

GamutX

Getting Started with GIT

GIT

Introduction to Version Control Systems (VCS)

- What is VCS ?
- What is Git?
- Examples of Popular VCS Tools
- What is GitHub?
- Purpose of GitHub

What is VCS ?

Imagine you're writing a book. 📖✍️



- ◆ Day 1: MyNovel.doc
- ◆ Day 2: MyNovel_v2.doc -----> Every New Changes
- ◆ Day 3: MyNovel_final.doc -----> Saved New Version
- ◆ Day 4: MyNovel_final_1.doc

! what if ?

- ✗ Forget which version is the latest?
- ✗ Want to go back to an old version?
- ✗ Accidentally delete an important part?

RESULT -----> • Messy file names | • No way to track changes | • Lost work if mistakes happen



Version Control System

A system for **tracking changes** to files and **managing versions over time**.

🔍 Without VCS:

- ◆ Day 1: MyNovel.doc
- ◆ Day 2: MyNovel_v2.doc
- ◆ Day 3: MyNovel_final.doc
- ◆ Day 4: MyNovel_final_1.doc

Hard to track changes!

🔧 With VCS:

- ✓ Day 1 - Initial draft saved
- ✓ Day 2 - Edited by Alex (Added Chapter 2)
- ✓ Day 3 - Edited by Lisa (Fixed typos)
- ✓ Day 4 - Reverted to previous version

Everything clean & organized!

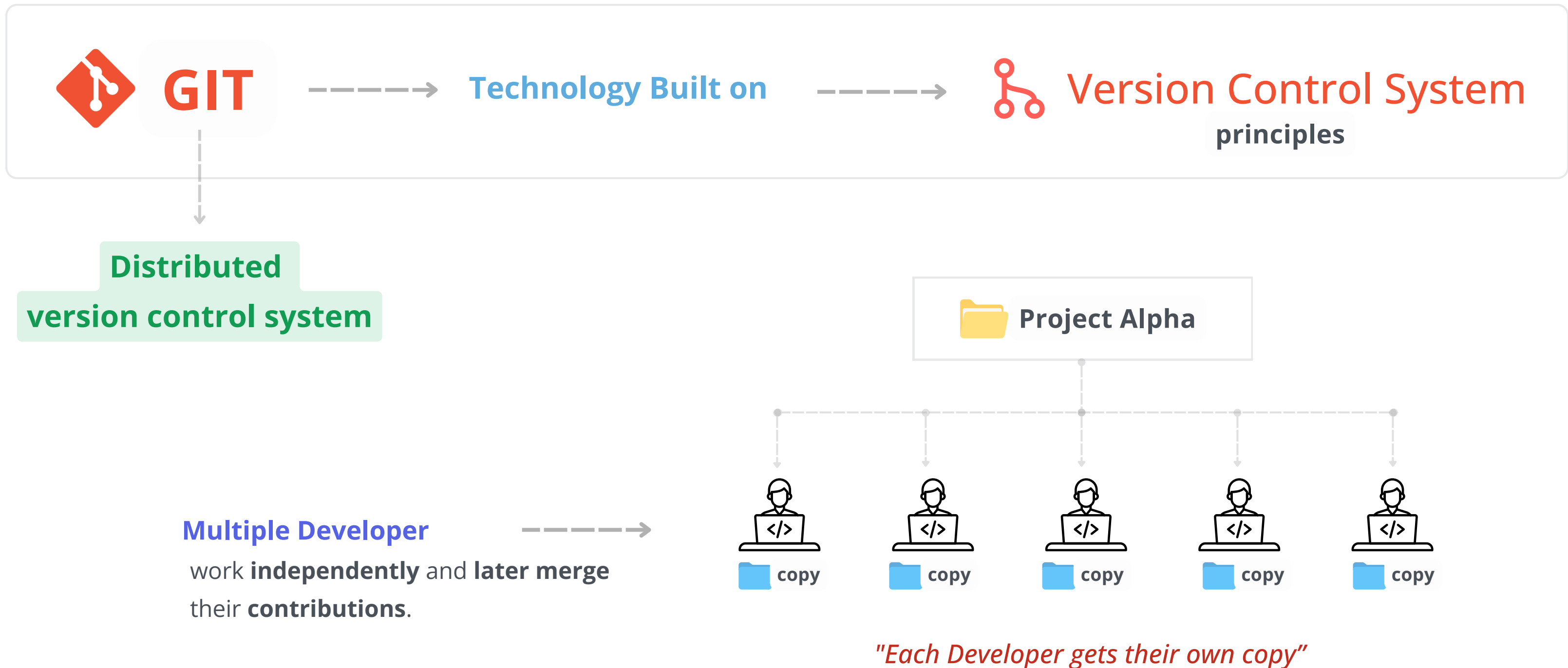
Key Features of VCS

- **Tracks File Changes**
 - Keeps a **record** of **every update**.
- **Collaborative Work**
 - **Teams can work together** smoothly.
- **Rollback Capability**
 - Easily **restore previous versions** if needed.
- **Branching & Merging**
 - **Work on different features separately** & merge later.

Why Use VCS?

- **Collaboration**
 - Developers **can work on the same project** without **overwriting** each other's work.
- **Error Recovery**
 - **Mistakes can be undone by rolling back** to a previous version
- **Change Tracking**
 - A full history of changes helps with **debugging** and **auditing**.

What is Git?



Key Features of Git

Distributed Architecture

Everyone gets a **full copy** of the project → **work offline**.

Branching & Merging

Experiment safely with **new features** & **bug fixes** without affecting **main code**.

Speed & Efficiency

Optimized for speed, **handling large projects with ease**.

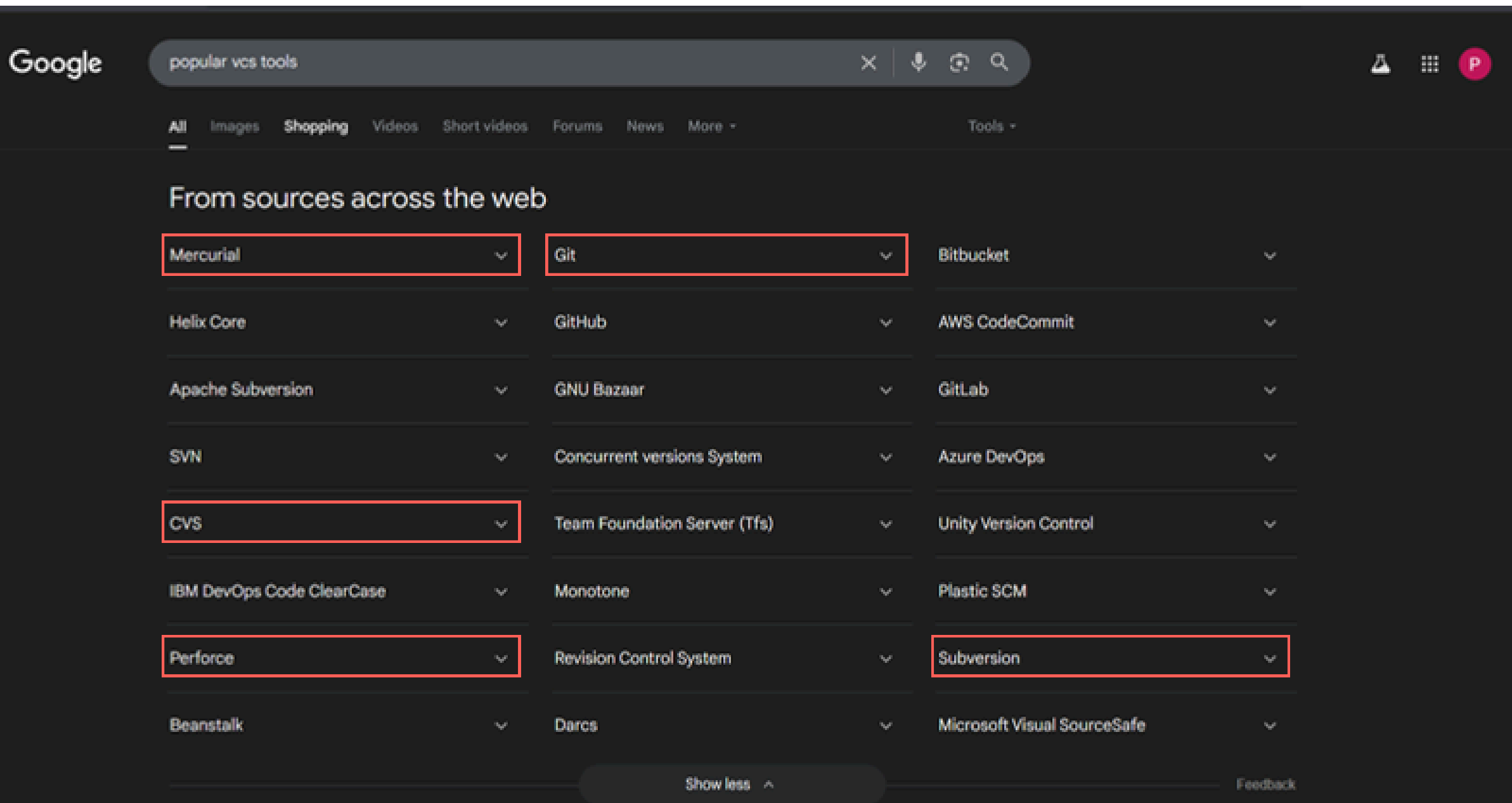
Data Integrity

Uses **cryptographic hash** functions (**SHA-1**) to ensure **data security**.



Examples of Popular VCS Tools

Let's See some **popular VCS Tools Available online**



 Subversion (SVN)

 Perforce

 Git

 Concurrent Versions System

 Mercurial

Concurrent Versions System

- One of the **earliest VCS tools**, often used for **academic projects**.

Perforce

- Used in **game development** and **large-scale projects**.

Git

- A **distributed VCS** (DVCS).
- The most widely used tool for **open-source** and **commercial projects**.

Mercurial

- Used in **large-scale software development**

Microsoft Team Foundation Server

- Now known as **Azure DevOps Server**.
- Integrates with Microsoft tools for **enterprise management**.



Subversion (SVN)

- **Open-source CVCS**.
- Popular for **enterprise software projects**

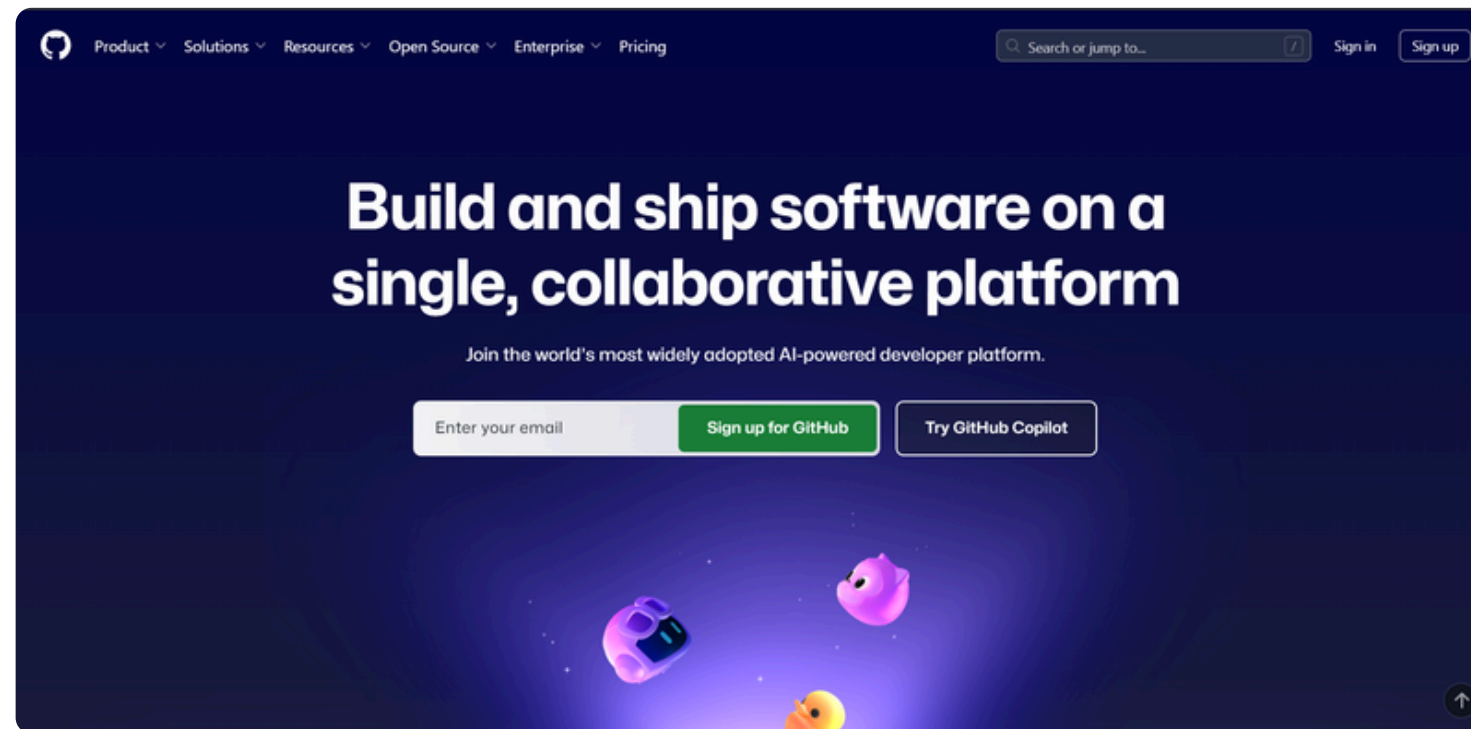
What is GitHub?

**GitHub**

A **platform** that **helps developers collaborate** using **Git**.



"Version Control System"



GitHub makes it easy to:

- ✓ Store your **code securely**.
- ✓ **Track changes** with Git's version control.
- ✓ Work with **multiple developers** on the **same project**.
- ✓ **Share** and **showcase projects** with the world.

"Think of GitHub as a social network for developers, where people share and contribute to code!"

Key Features of GitHub

- **Version Control**

Uses **Git** to track changes of projects.

- **Collaboration**

Enables **teams to work together** on **same projects**.

- **Community**

hosts **millions** of **open-source projects** and fosters a community of developers contributing to various projects.

- **Integration**

Supports **integration** with

- **Continuous Integration (CI),**
- **Continuous Deployment (CD),**

and other **DevOps tools**.

Purpose of GitHub

GitHub serves multiple purposes:

- **Team Collaboration**
 - Developers can work together on the same codebase efficiently.
 - **Project Management**
 - Track issues, assign tasks, and manage workflows.
 - **Open Source Contributions**
 - Developers can contribute to projects and improve software used by millions.
 - **Portfolio Building**
 - Developers use GitHub to showcase their work and improve their career opportunities.
- 

GitHub vs Git: What's the Difference?

Feature	Git	GitHub
• Type	→ A version control system	→ A platform built on Git
• Purpose	→ Tracks changes in code	→ Stores & manages Git repositories online
• Works Offline?	→ ☒ Yes	→ ✗ No, requires internet for full functionality
• Collaboration	→ Local, requires manual sharing	→ Built-in collaboration tools

- So, while **Git is the tool that manages version control**,
- **GitHub** is the place where **developers store and collaborate on Git projects**.

Getting Started with GitHub

- Creating a GitHub Account
- UI Overview of GitHub
- Creating Your First Repository
- What is a Repository?
- Public vs. Private Repositories
- Searching Public Repositories on GitHub
- Why GitHub is Important

Creating a GitHub Account

Go to GitHub's Website

GamutX

- 🔍 Open a web browser.
- 🌐 Search "**GitHub**" in Google.
- 🔗 Click on the **first link** ([GitHub](#)).

GitHub Copilot is now available for free. [Learn more](#)

 [Product](#) [Solutions](#) [Resources](#) [Open Source](#) [Enterprise](#) [Pricing](#)

Build and ship software on a single, collaborative platform

Join the world's most widely adopted AI-powered developer platform.

[Sign up for GitHub](#)[Try GitHub Copilot](#)

Sign Up for an Account & Verify It

Sign up to GitHub

Email*

fohewo9362@oziere.com

Password*

Password

Password should be at least 15 characters OR at least 8 characters including a number and a lowercase letter.

Username*

Username

Username may only contain alphanumeric characters or single hyphens, and cannot begin or end with a hyphen.

Continue >

By creating an account, you agree to the [Terms of Service](#). For more information about GitHub's privacy practices, see the [GitHub Privacy Statement](#). We'll occasionally send you account-related emails.

Sign Up for an Account

- Click "**Sign Up**" on the GitHub homepage.
- Enter your **username, email, and password**.
- Click "**Next**" to continue.
- Complete the **email confirmation**.
- Solve the **CAPTCHA verification** (if required)

💡 Important:

- Choose a **professional email ID** and a simple,
- **easy-to-remember username** for **GitHub account**.

✓ Good Example:

- ✓ Email: **johndoe@gmail.com**
- ✓ Username: **johndoe** or **john-doe**

✗ Avoid:

- ✗ Email: **coolhacker99@hotmail.com**
- ✗ Username: **John-Mighty**

Set Up Your Profile

Add

- bio, profile picture, and other details. Personalize your account to stand out!

The screenshot shows a GitHub profile page for a user named 'Rahul-Duaaa'. The browser tabs at the top include 'Rahul-Duaaa' and 'Temp Mail - Disposable Temporary'. The address bar shows 'github.com/Rahul-Duaaa'. The profile header includes the username 'Rahul-Duaaa', a search bar, and navigation links for Overview, Repositories, Projects, Packages, and Stars. The main content area features a large circular profile picture placeholder with a red and white cross pattern. Below the picture is a 'Change your avatar' button. To the right of the picture, it says 'You don't have any public repositories yet.' and 'Popular repositories'. Below the profile picture, it says 'Rahul-Duaaa' and 'Joined 7 minutes ago'. To the right of the profile picture, there is a '1 contribution in the last year' section with a 'Contribution settings' dropdown and a '2025' button. Below this is a contribution graph showing a single green square on March 1st. The graph is titled '1 contribution in the last year' and includes a legend for 'Learn how we count contributions' and a color scale from 'Less' to 'More'. Below the graph, there is a text box explaining the contribution graph and a button to 'Read the Hello World guide'. The footer of the page shows the URL 'https://github.com/account'.

Popular repositories

You don't have any public repositories yet.

1 contribution in the last year

Contribution settings

2025

Mar Apr May Jun Jul Aug Sep Oct Nov Dec Jan Feb Mar

Mon

Wed

Fri

Learn how we count contributions

Less More

This is your **contribution graph**. Your first square is for joining GitHub and you'll earn more as you make [additional contributions](#). More contributions means a higher contrast square color for that day. Over time, your chart might start looking [something like this](#).

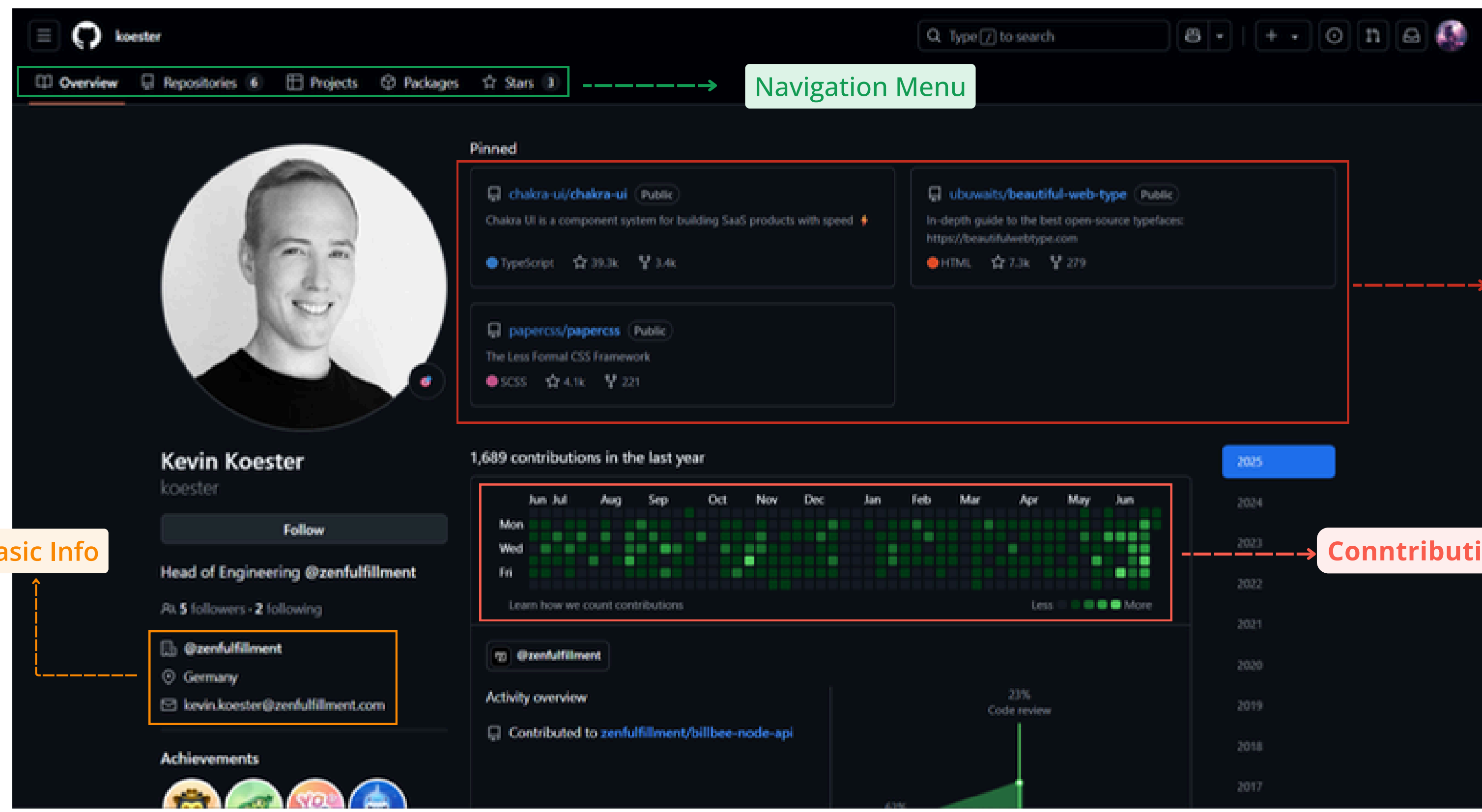
We have a quick guide that will show you how to create your first repository and earn more green squares!

[Read the Hello World guide](#)

Contribution activity

UI Overview of GitHub

GamutX



Basic Info

Navigation Menu

Repositories

Contribution Graph

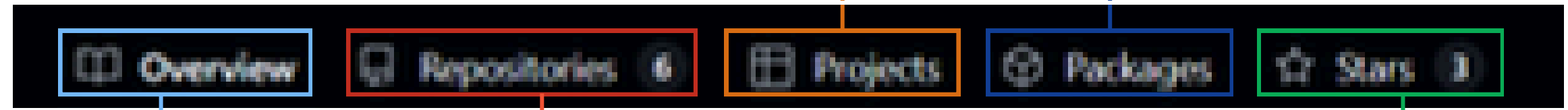
Navigation Menu

Projects

Boards to manage tasks and workflows.

Packages

Stored software packages or libraries.



Overview

Summary of your GitHub activity and profile

Repositories

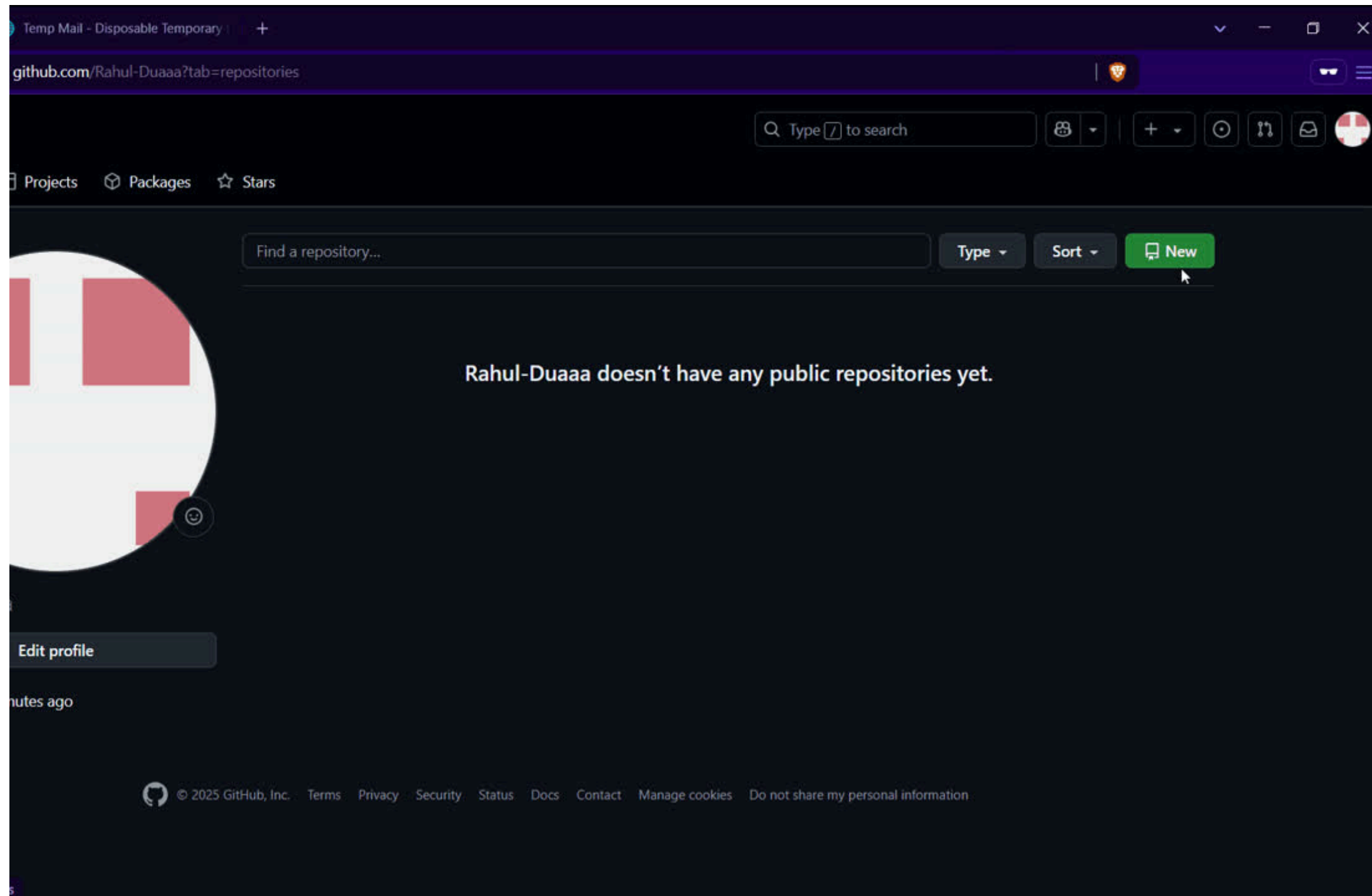
List of your code projects.

Stars

Repos you've liked or bookmarked.

Creating Your First Repository

GamutX



- Go to github.com and log in
- Click your **profile picture** → **Your repositories**
- **Click** the green **“New”** button
- Set **repository name** (e.g., hello)
- Add a **short description** (optional)
- **Select Public**
- Click **“Create repository”**

Naming Rules for Repositories

❌ **Avoid spaces** in repository names.

Instead, use:

✅ **Hyphens** (-) → my-project

✅ **Underscores** () → my_project

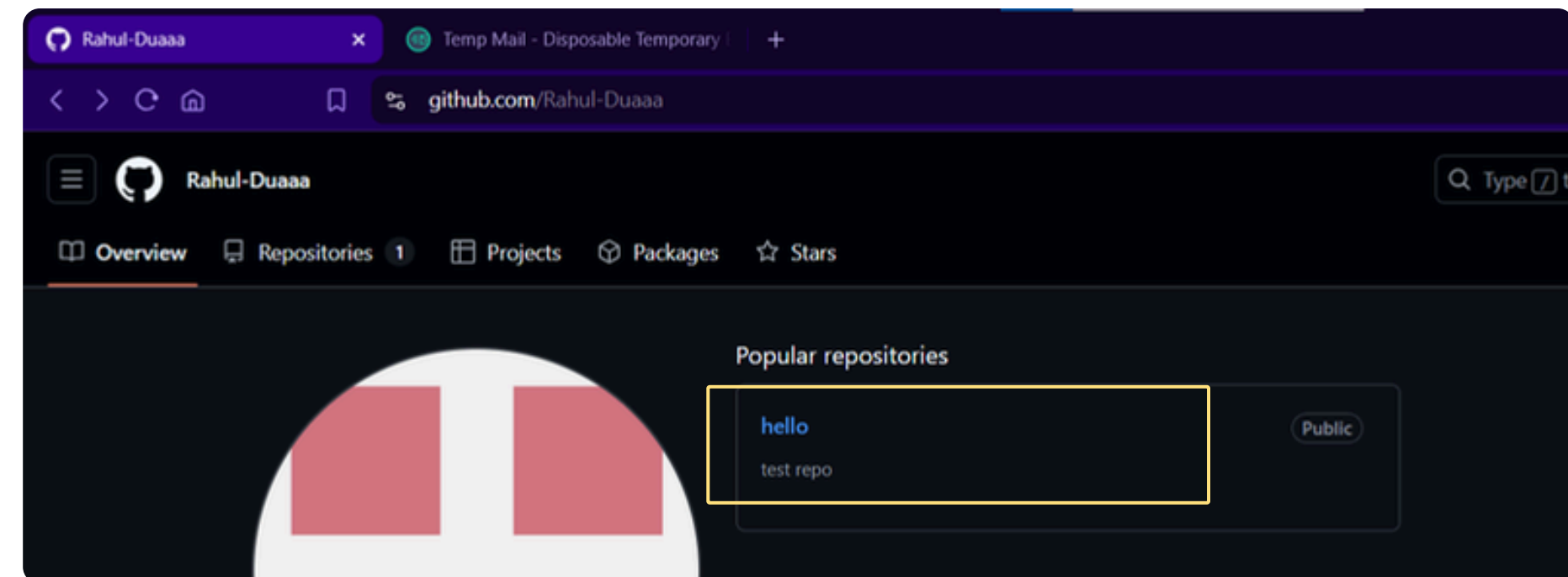
What is a Repository?

📁 Repository

- A repository (repo) is a storage space **where a project's code, files, and version history are kept.**
- Repositories make it easier to track changes, collaborate, and manage versions efficiently.

A repository can contain:

- **Code**
Source code, scripts, and configuration files.
- **Version History**
A complete history of file changes, tracked by Git.
- **Documentation**
README files, wikis, and project guides.
- **Branches**
Independent versions of the project for working on different features or bug fixes.



Understanding Public vs Private Repository

Public

- **Anyone can view** and **contribute**.
- Best for **open-source projects**.

Private

- Only **visible to you** and **selected collaborators**.
- Best for **personal projects**.

Public vs. Private Repositories

GitHub offers two types of repositories: **Public** and **Private**.

Public Repository

◆ Characteristics:

- **Visible to everyone.**
- Open to **public contributions**.
- Best for **open-source projects, code sharing**

✓ Advantages:

- ✓ Encourages **community contributions**.
- ✓ Increases project **visibility**.
- ✓ Promotes **transparency** and **trust**.

⚠ Disadvantages:

- ✗ Risk of **intellectual property theft**.
- ✗ Not suitable for **sensitive** or **confidential data**.

Private Repository

◆ Characteristics:

- ✓ Visible only to **authorized users**.
- ✓ **Access** controlled by the **repository owner**.
- ✓ Best for **proprietary software & internal development**.

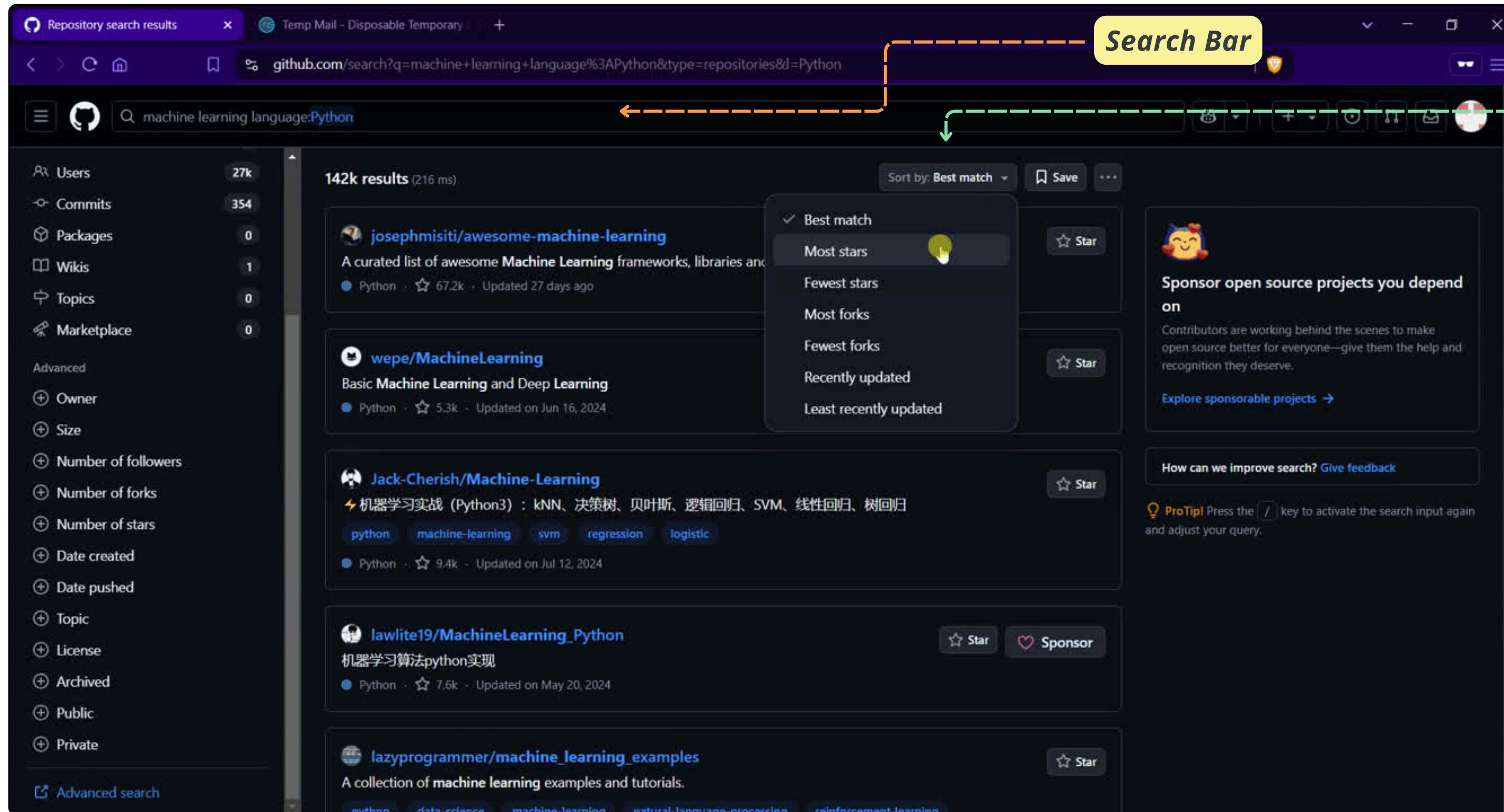
✓ Advantages:

- ✓ Enhanced **security & confidentiality**.
- ✓ **Full control** over who can **access** and **contribute**.
- ✓ **Protects sensitive information** & private code

⚠ Disadvantages:

- ✗ **Only authorized users** can contribute.
- ✗ **Less visibility** compared to **public repositories**.

Exploring GitHub Repositories



Filter search based on Stars,

You can Filter programming language from here



HO: Create a GitHub Repository



Objective

- Learn how to create a new repository on GitHub.



Instructions:

Follow these tasks to complete your first GitHub repository setup:

- Log in to your GitHub account.
- Create a **new repository** with a name like **my-first-repo**.
- Add a short **description** to tell what this repo is about.
- Set the repository to either **Public** or **Private**, based on your preference.
- **Initialize** the repository with a **README** file.
- Click **Create repository**.
- Finally, **verify** the repository appears on your profile under the **Repositories** tab.

 Steps to Create a Repository:

1. Open [GitHub](#) in your web browser and **log in**.
2. Click the **+** icon in the top-right corner of the page.
3. Choose **New repository** from the dropdown.
4. Fill in the required details:
 - **Repository name:** e.g., my-first-repo
 - **Description:** (Optional) e.g., This is my first GitHub repository
 - **Visibility:** Choose **Public** or **Private**
 - Check the box **"Initialize this repository with a README"**
5. Click **Create repository**.
6. You'll be redirected to your new repo's page. Confirm it's visible in your GitHub **profile** under **Repositories**.

Why GitHub is Important

✓ Global Collaboration

Work with **developers worldwide**

Contribute to **open-source** & **private projects**

✓ Version Control & Backup

Secure **cloud storage** for your code

Restore previous versions anytime

✓ Integration with DevOps Tool

Connect with **CI/CD tools** for **automation**

Supports **integration** with tools like **Jenkins**, **GitHub Actions**, and **Docker**.

✓ Open-Source Ecosystem

Learn from **high-quality projects**

Improve **coding skills** by **contributing**

✓ Visibility

Showcase your **skills** with a **strong GitHub profile**

Open-source contributions can boost **job opportunities**



Ways of Cloning a Repositories

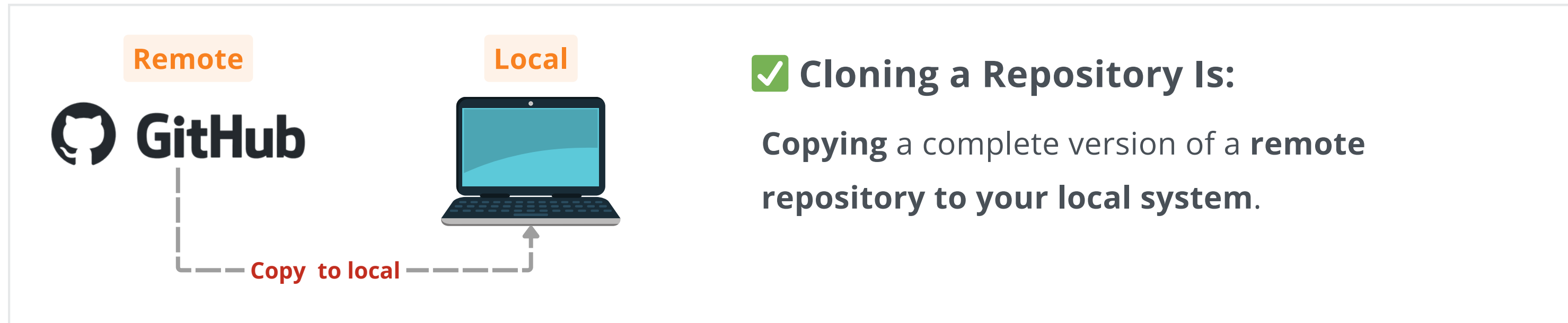
- GitHub allows users to interact with repositories using HTTP or SSH.

Method	Best For	Pros	Cons
• HTTP	→ Beginners or infrequent use	→ Easy to set up, no extra keys needed	→ Requires login for every push/pull
• SSH	→ Frequent users, secure access	→ More secure, seamless authentication	→ Requires SSH key setup

Cloning a Repository

- What Does Cloning a Repository Mean?
- Git Configuration
- How to Clone Repo from Remote to Local
- What Happens During Cloning?
- Understanding Local and Remote Repositories

What Does Cloning a Repository Mean?



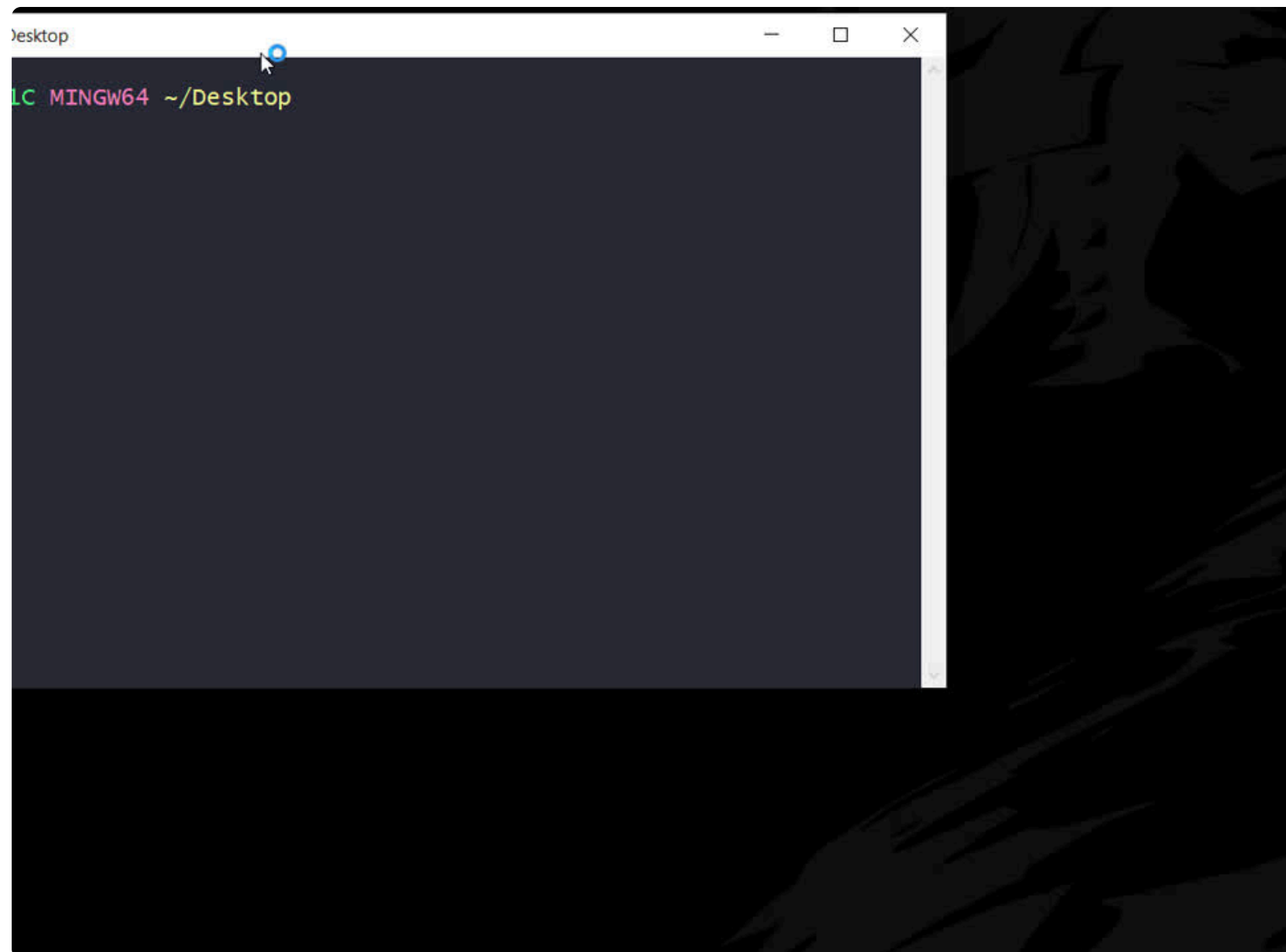
◆ Why Clone a Repository?

- **Access full project history** – Includes commits & branches
- **Work offline** – No internet needed to code & test
- **Make & test changes** – Push updates when ready
- **Collaborate easily** – Work with your team on the same codebase

Git Configuration

- Before cloning a repository, Let's Configure Git on Local System

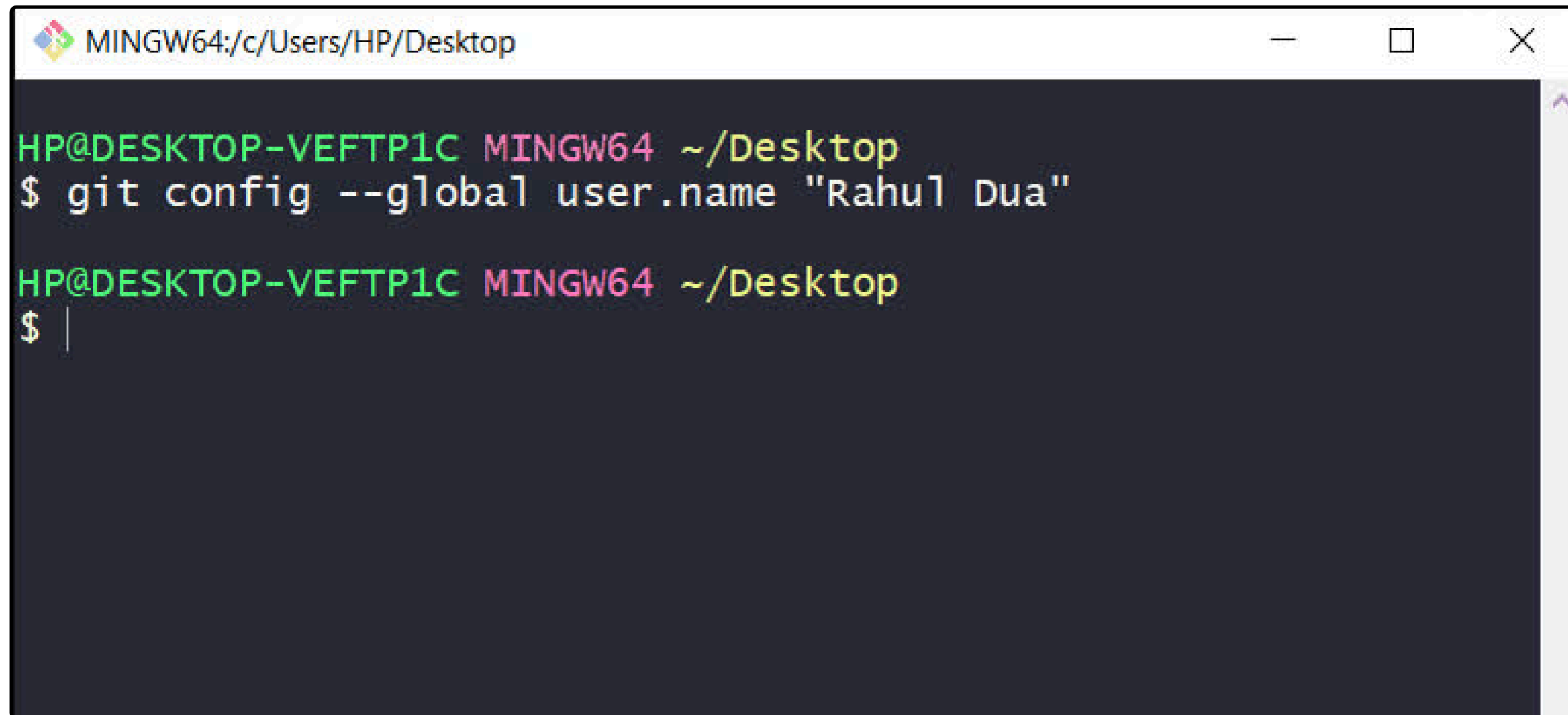
Open Git Bash



- **Right-click** anywhere on your **desktop** or **inside a folder**.
- Click on "**Git Bash Here**" to open the terminal.

Configure Your Name & Email

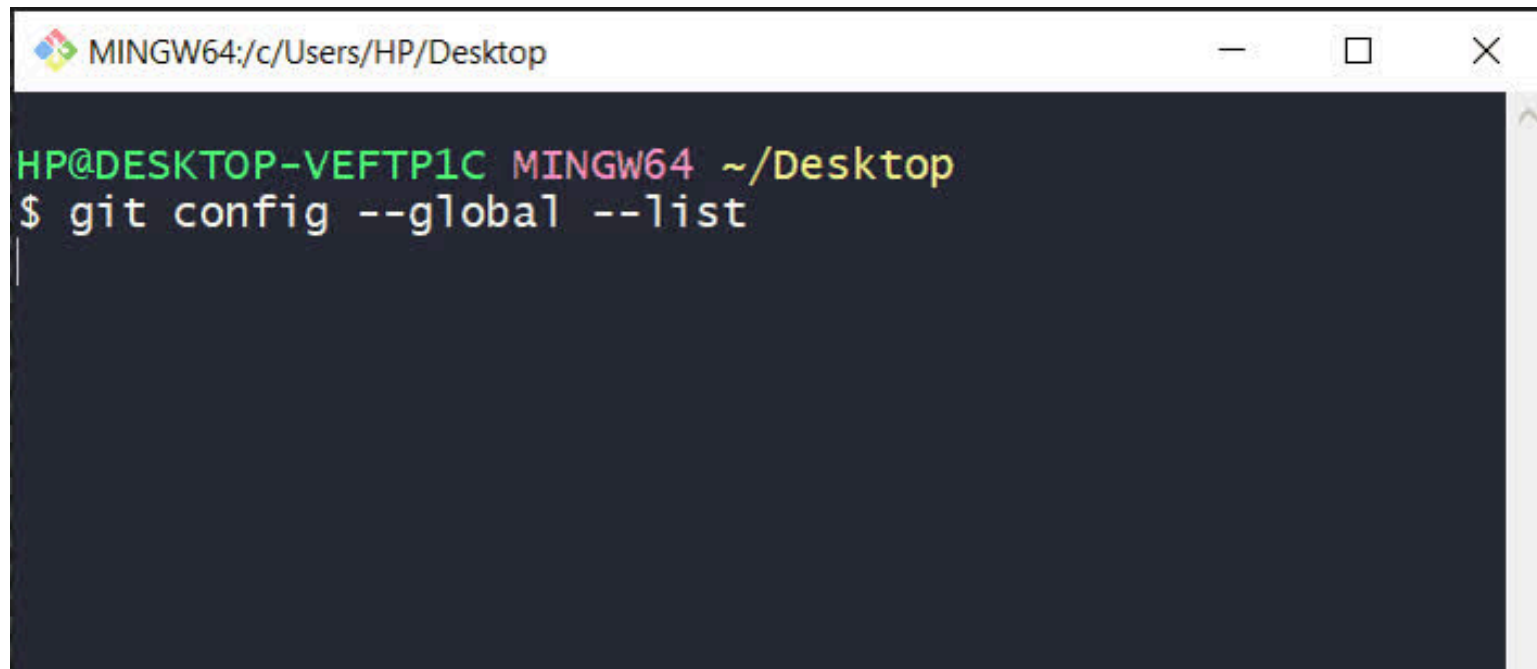
In Git Bash, run the following commands



```
MINGW64:/c/Users/HP/Desktop  
HP@DESKTOP-VEFTP1C MINGW64 ~/Desktop  
$ git config --global user.name "Rahul Dua"  
HP@DESKTOP-VEFTP1C MINGW64 ~/Desktop  
$ |
```

✓ This applies to all repositories on your system.

Verify Your Configuration

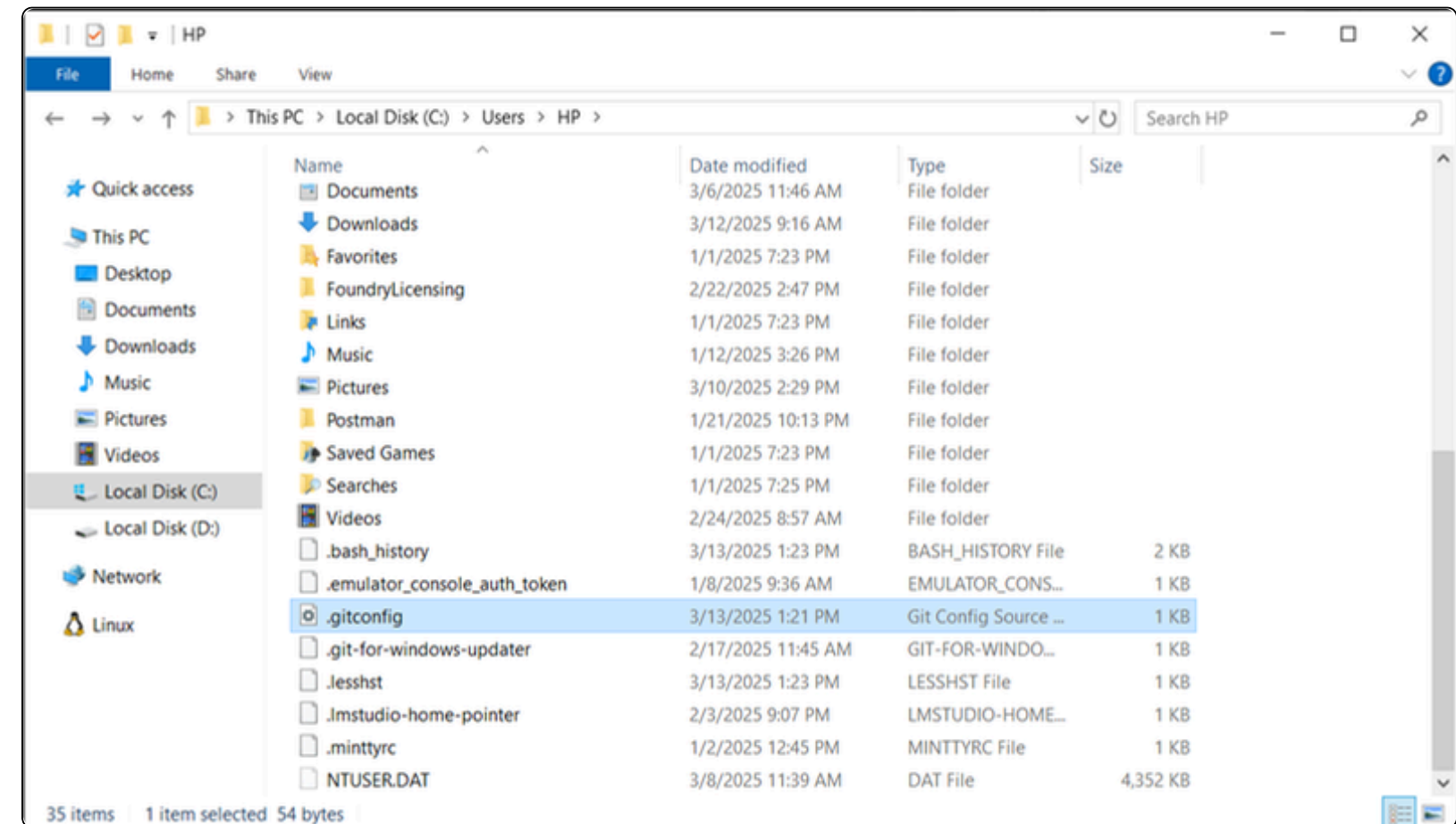


```
MINGW64:/c/Users/HP/Desktop
HP@DESKTOP-VEFTP1C MINGW64 ~/Desktop
$ git config --global --list
```

Type --> **git config --global --list**

✓ **This will show your configured name and email.**

Where is the Configuration Stored?



On Windows: **C:\Users\<YourName>\.gitconfig**

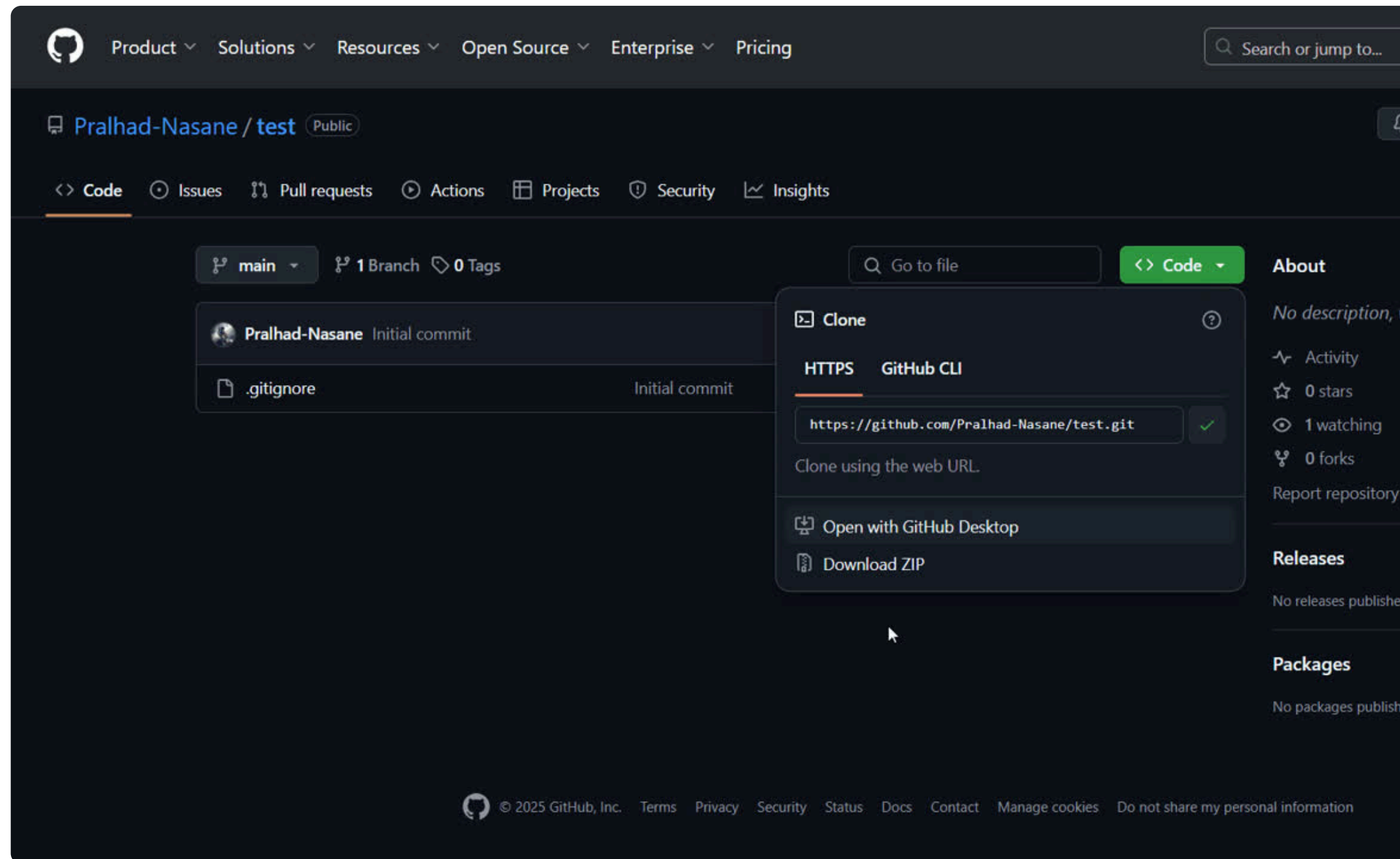
On macOS/Linux: **~/.gitconfig**

How to Clone Repo from Remote to Local

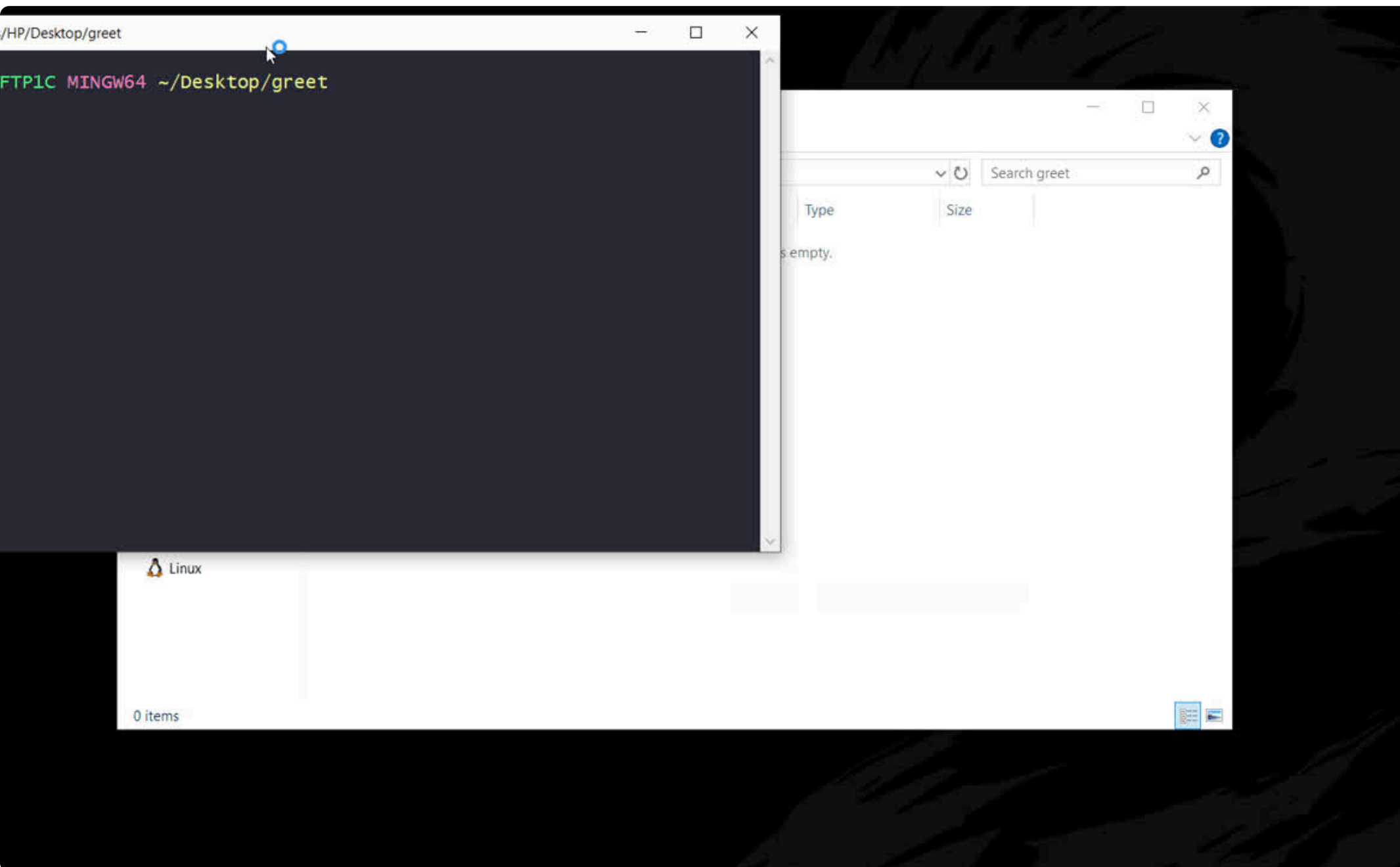
Copy Repository Link from GitHub

Go to **GitHub** and **open your repository**.
Click on the **Code** button (**Green Button**).
Copy the **HTTPS** URL

<https://github.com/username/repository-name.git>



Clone Repository to Local Folder



Open or **create a folder** where you want to store the **cloned repository**.

Right-click inside **folder** → Click "**Open Git Bash Here**".

In **Git Bash**, run the following **command**:

git clone <repository_link>

Paste the copied repository link using:

- Windows/Linux: **Press Shift + Insert**
- macOS: **Press Command (⌘) + V**

Git will download the repository into your local folder.

What Happens During Cloning?

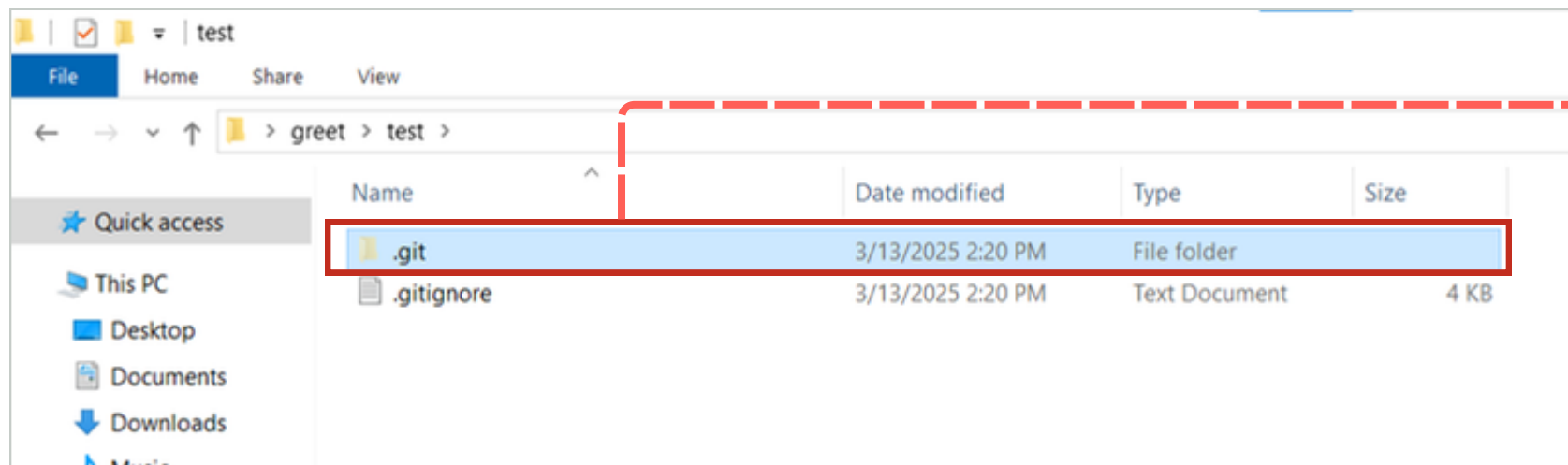
When you clone a repository, Git copies the entire project from GitHub to your local machine.

```
MINGW64/c/Users/HP/Desktop/greet
HP@DESKTOP-VEFTP1C MINGW64 ~/Desktop/greet
$ git clone https://github.com/Pralhad-Nasane/test.git
Cloning into 'test'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
```

- ✓ **All Project Files** – Includes code, images, documentation, etc.
- ✓ **A Hidden .git Folder** – Stores everything Git needs for version control.

What is the .git Folder?

When you clone a repository, a hidden .git folder is created inside the cloned directory. This folder contains:



- **Commit History** – Tracks all past changes & versions.
- **Branches & Tags** – Manages different versions of the project.
- **Configuration Files** – Stores repository-specific settings.

Understanding Local and Remote Repositories

Git works with two types of repositories:

Local Repository

- Stored on your **computer**
- Allows you to make **changes, commit updates**, and **work offline**
- **Changes** stay **private** until **pushed to a remote repository**

Remote Repository

- Hosted on **GitHub** or another **Git server**
- **Acts as a central hub** where **multiple developers collaborate**
- **Local changes must be pushed** to the remote repository for others to see

Git Workflow

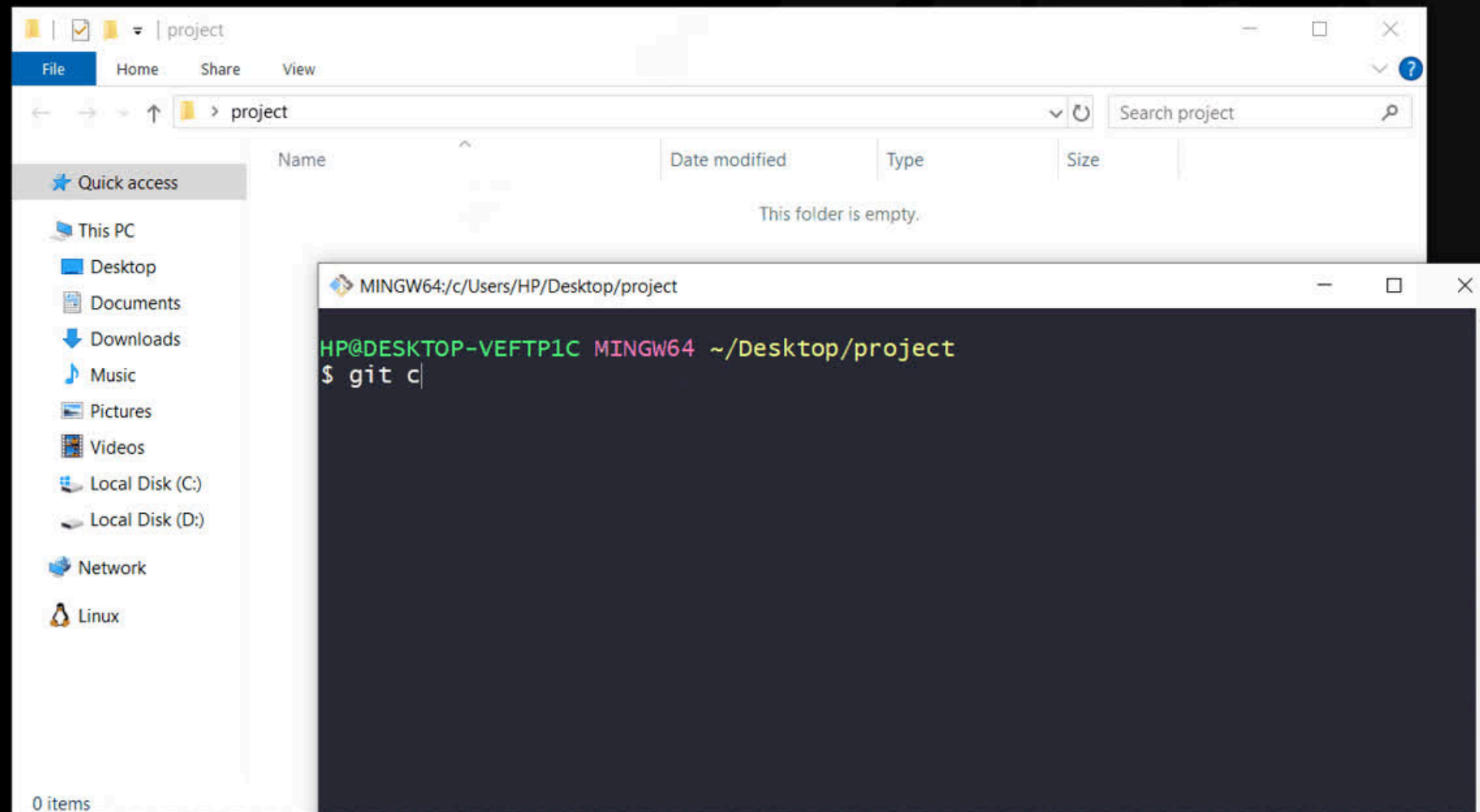
- Git Workflow Using VS Code
- Understanding Staging & Committing

Git Workflow Using VS Code

Steps in the Git Workflow

- ✓ **Step 1:** Cloning a Repository Using Git Bash
- ✓ **Step 2:** Loading the Project in VS Code
- ✓ **Step 3:** Making Changes
- ✓ **Step 4:** Reviewing Changes
- ✓ **Step 5:** Staging Changes
- ✓ **Step 6:** Committing Changes
- ✓ **Step 7:** Pushing Changes to GitHub

Cloning a Repository Using Git Bash



Copy the Repository Link from GitHub

- Go to the GitHub repository.
- Click the **green "Code" button**.
- Click "**Copy**" to get the repository URL.

Open Git Bash

- Windows: **Right-click** inside the **folder** you want
- Select "**Open Git Bash Here**"

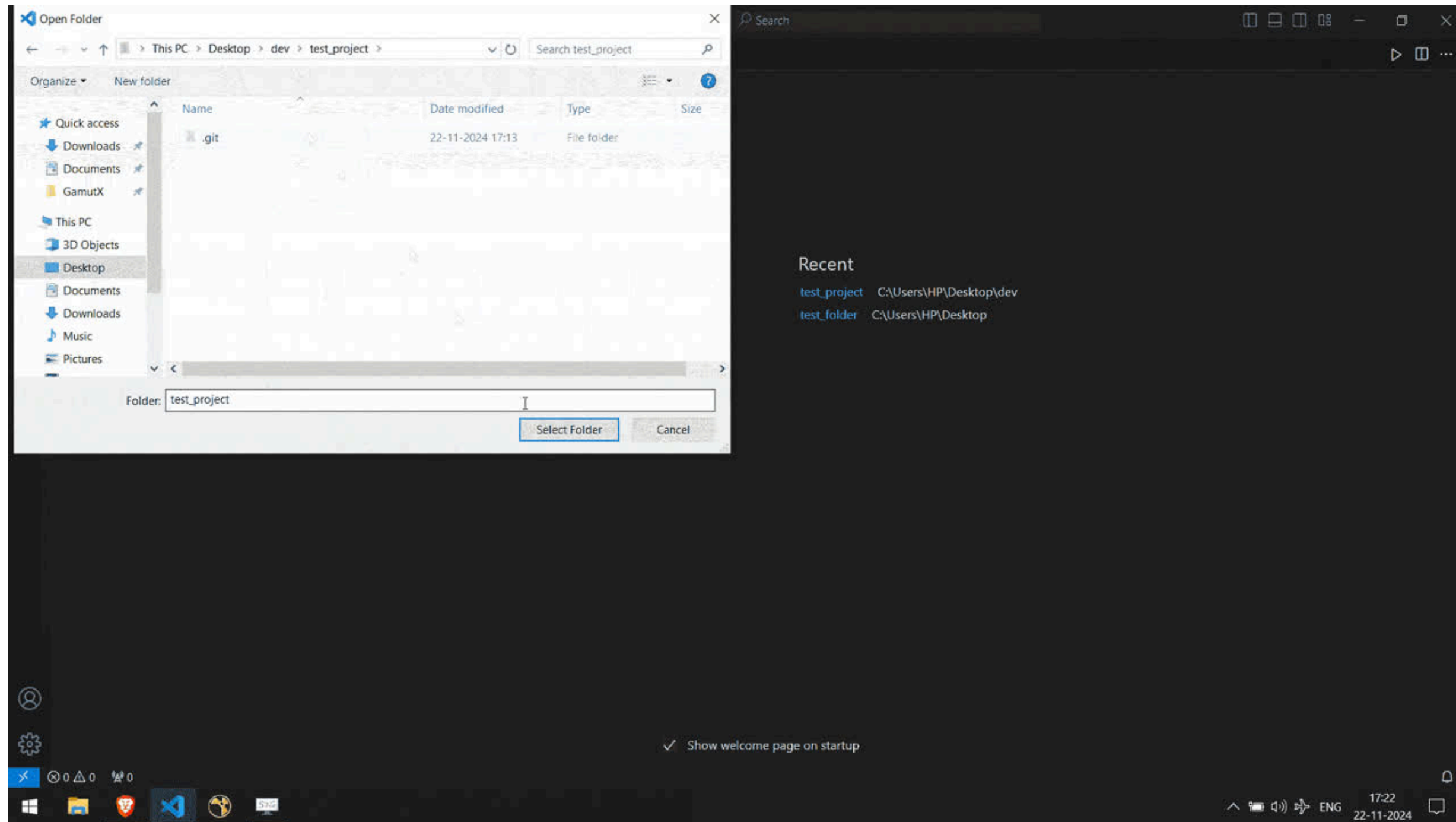
Run the Clone Command

- **git clone <repository_url>**

📌 Example:

- **git clone** <https://github.com/Pralhad-Nasane/test.git>

Loading the Project in VS Code



Open **VS Code**

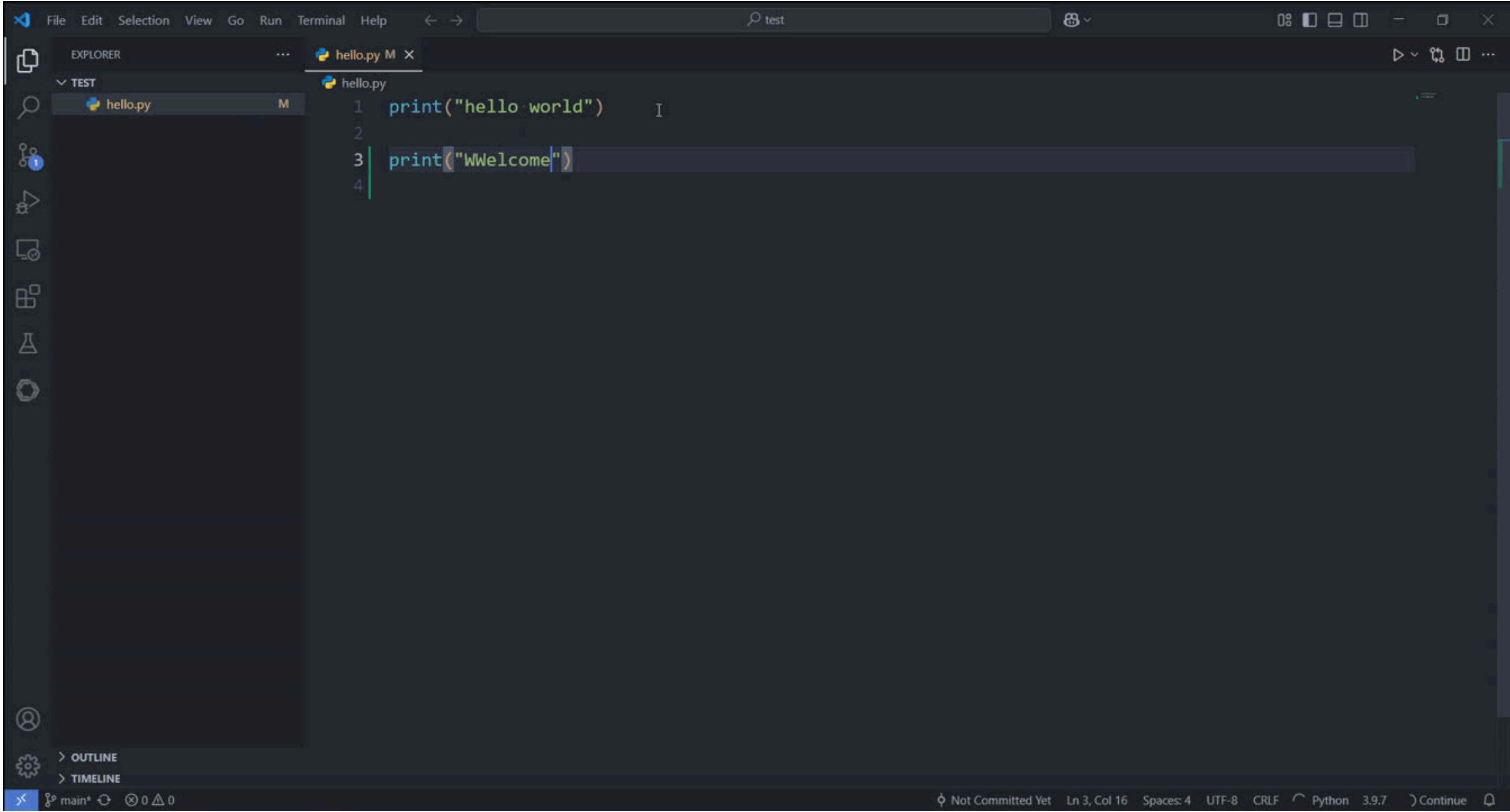
Go to **File > Open Folder** 

Select the **cloned repository folder** & **open it**

Making Changes

GamutX

- Open **hello.py** in VS Code
- Modify the file – Add/edit content.
 - **print("Welcome")**
- Save the file (**Ctrl + S / Cmd + S**).



The screenshot shows the Visual Studio Code interface with a dark theme. The Explorer sidebar on the left shows a file named 'hello.py' under a 'TEST' folder. The main editor area displays the contents of 'hello.py'. The first line is `print("hello world")`. The second line is empty. The third line is `print("WWelcome")`, with the text highlighted in blue. The status bar at the bottom indicates the file is not committed, the cursor is at line 3, column 16, and the file is encoded in UTF-8 with CRLF line endings. The Python version 3.9.7 is also shown.

```
1 print("hello world")
2
3 print("WWelcome")
4
```

Reviewing Changes

VS Code interface showing the process of reviewing changes in a file named `hello.py`.

The interface is split into two panels:

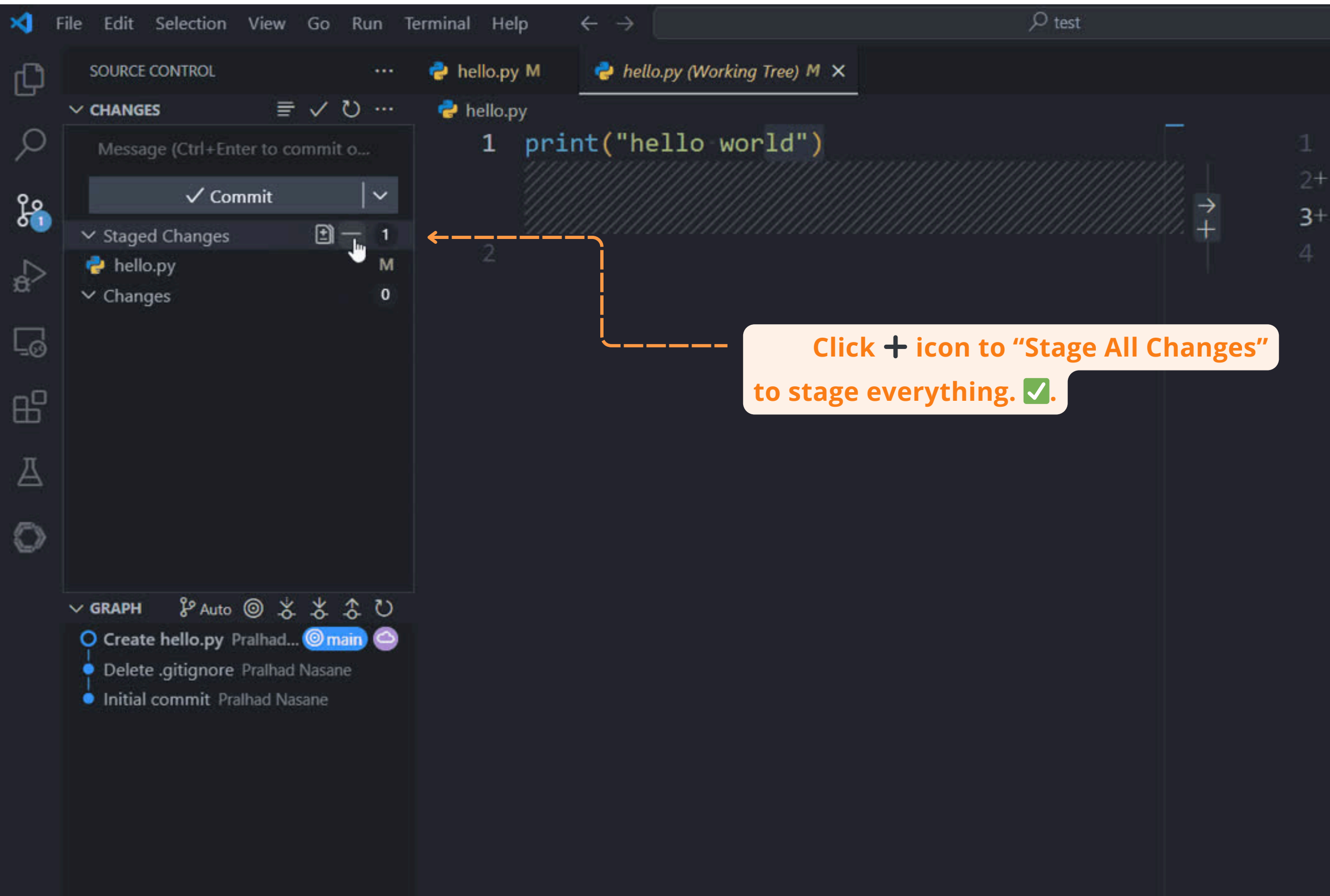
- Before Changes:** Shows the original code: `1 print("hello world")`.
- After Changes:** Shows the modified code: `1 print("hello world")` followed by a new line `2+ print("WWelcome")`. The new line is highlighted in green, indicating a change.

Annotations and actions:

- Click Source Control icon in VS Code (Ctrl + Shift + G).** (Indicated by a red dashed arrow pointing to the Source Control icon in the sidebar).
- Click on the Modified file** (Indicated by an orange dashed arrow pointing to the `hello.py` file in the CHANGES list).
- Green = Changes Made** (Indicated by a green dashed arrow pointing to the green highlight in the 'After Changes' panel).

Staging Changes

GamutX



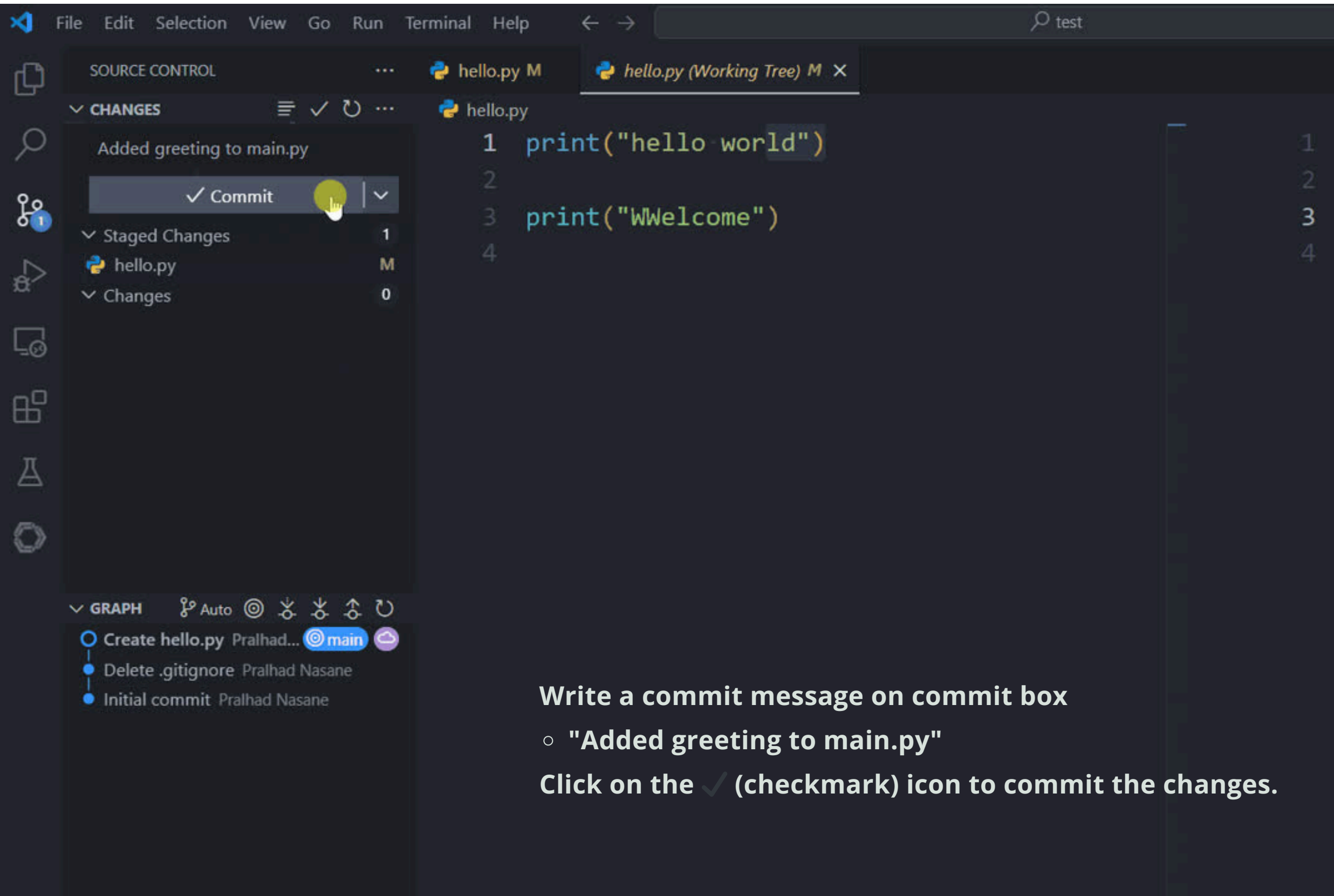
Click + icon to "Stage All Changes" to stage everything. ✓.

What is Staging means?

✓ Staging means preparing your changes before committing them.

Committing Changes

GamutX



Write a commit message on commit box

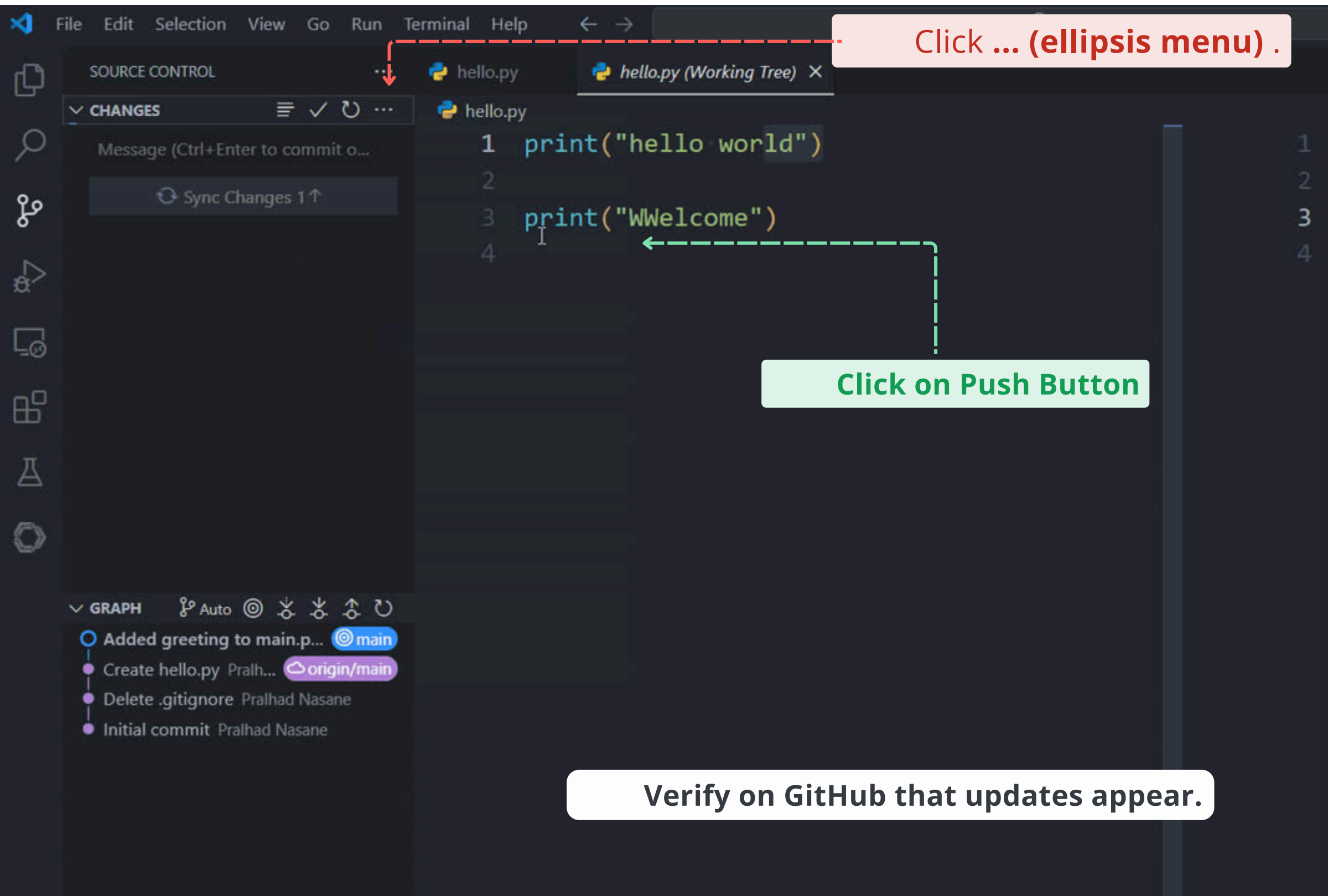
- "Added greeting to main.py"

Click on the ✓ (checkmark) icon to commit the changes.

📌 What is a Commit means ?

- ✓ A commit is a record of changes made at a specific point in time.

Pushing Changes to GitHub

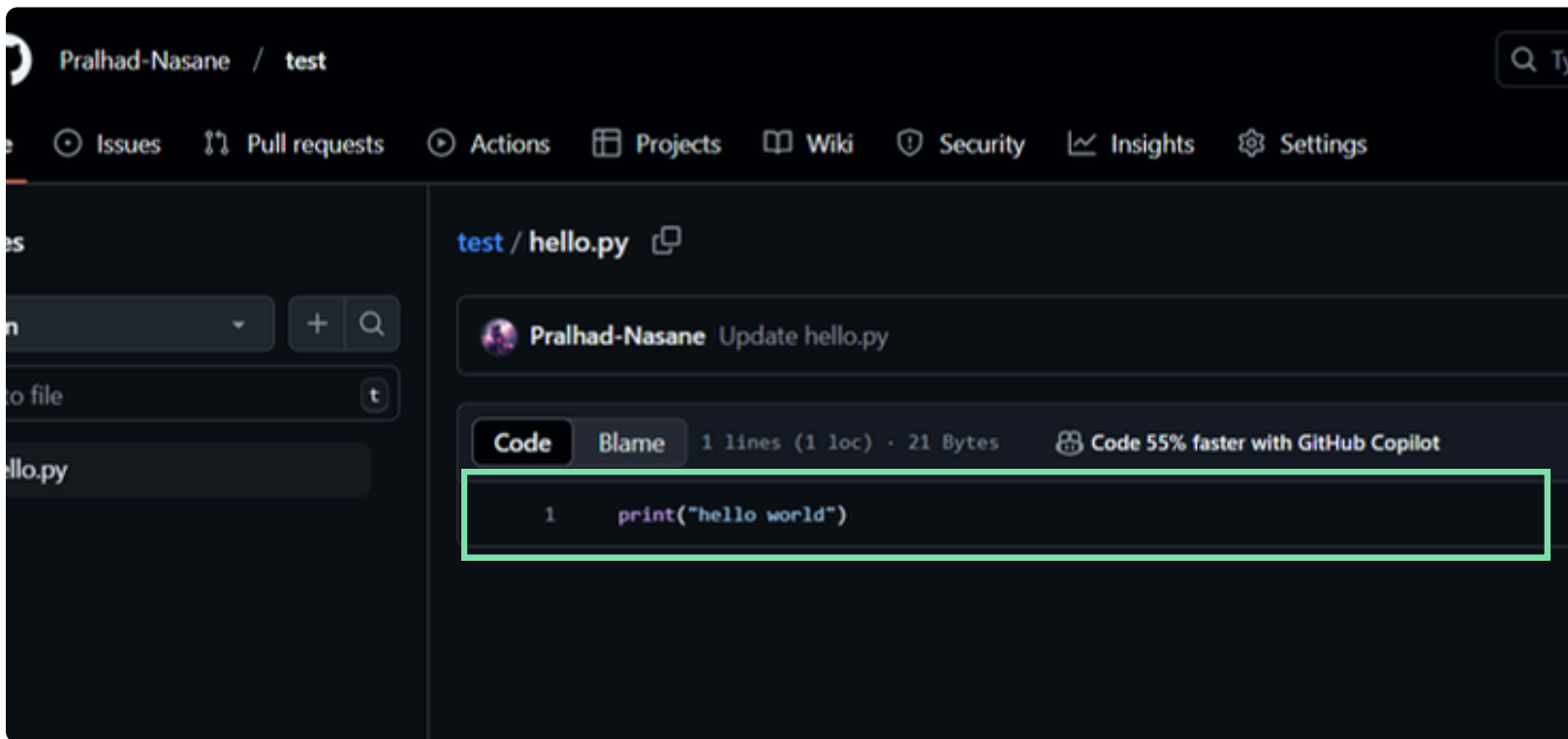


Note

- If it's your first time pushing to this repository,
- GitHub might prompt you to authorize VS Code.

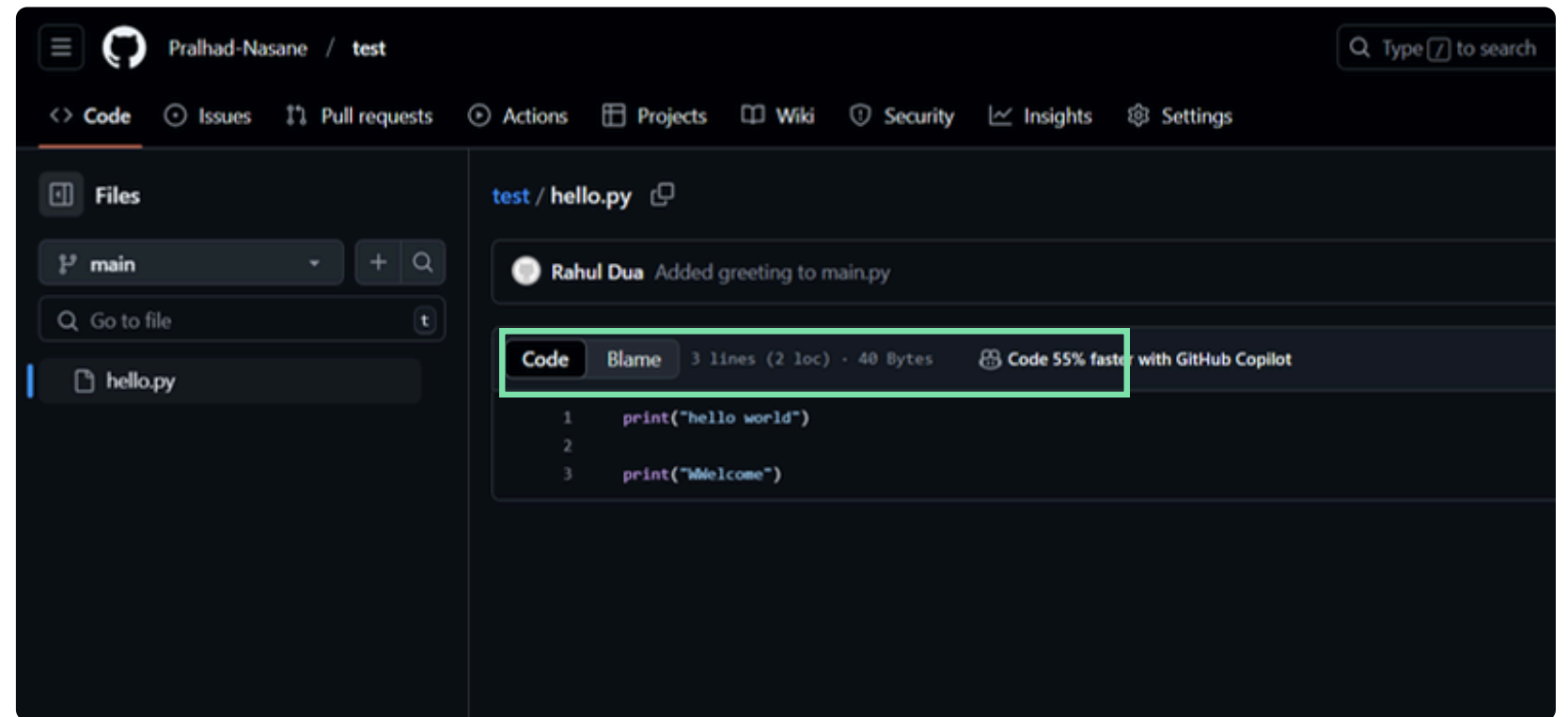
Viewing Changes on Remote

◆ Before Doing Changes



The screenshot shows the GitHub interface for a repository named 'test' by user 'Pralhad-Nasane'. The file 'test / hello.py' is selected. The commit history shows a single commit by 'Pralhad-Nasane' titled 'Update hello.py'. The code view shows a single line of Python code: `1 print("hello world")`. The interface includes navigation tabs for Issues, Pull requests, Actions, Projects, Wiki, Security, Insights, and Settings. The file view includes tabs for Code and Blame, and a status bar indicating '1 lines (1 loc) · 21 Bytes' and 'Code 55% faster with GitHub Copilot'.

◆ After Pushing Changes



The screenshot shows the GitHub interface for the same repository 'test' by user 'Pralhad-Nasane'. The file 'test / hello.py' is selected. The commit history shows a commit by 'Rahul Dua' titled 'Added greeting to main.py'. The code view shows three lines of Python code: `1 print("hello world")`, `2`, and `3 print("Welcome")`. The interface includes navigation tabs for Code, Issues, Pull requests, Actions, Projects, Wiki, Security, Insights, and Settings. The file view includes tabs for Code and Blame, and a status bar indicating '3 lines (2 loc) · 40 Bytes' and 'Code 55% faster with GitHub Copilot'.



H0. Practicing GIT Workflow



Objective

- Practice **creating files, staging changes, committing, and pushing** to a GitHub repository.



Instructions:

Follow these tasks to practice using Git with your GitHub repository:

- In your cloned GitHub repo folder, **create two Python files**:
 - **file1.py**: Print *Hello from file1!*
 - **file2.py**: Print *Hello from file2!*
- Use the **Source Control** tab in VS Code to:
 - **Stage the files** (prepare them for commit).
 - **Commit** the changes with a meaningful message (e.g., Added file1.py and file2.py).
- **Push** your committed changes to the GitHub repository.
- **Check GitHub to confirm your files appear after pushing**

 Steps to Complete the Task

1- Create Python Files:

- Open your cloned repo folder in **VS Code**.
- Create **file1.py** and add this code:

```
print("Hello from file1!")
```

- Create **file2.py** and add this code:

```
print("Hello from file2!")
```

2- Stage Files:

- Go to the **Source Control** tab in VS Code (icon that looks like a branch).
- You'll see your file changes under Changes.
- Click the **+ icon** next to each file to stage them—or click + beside Changes to stage all.

3- Commit Changes:

- In the message box at the top, type a commit message like:

```
Added file1.py and file2.py
```

- Click the ✓ **checkmark** to commit.

4- Push to GitHub:

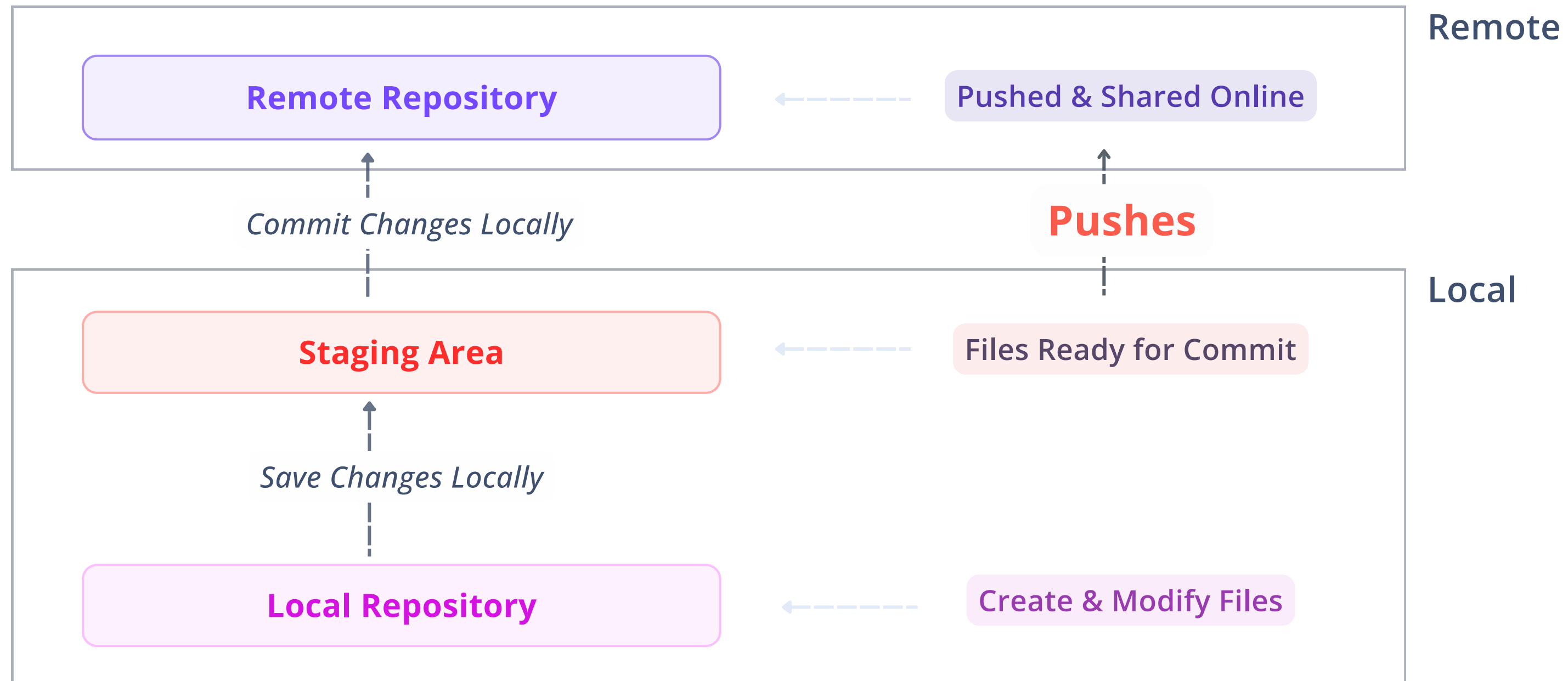
- Click the **three-dot menu (...)** in the Source Control tab.
- Choose **Push**.
- VS Code will upload your changes to GitHub.

5- Verify:

- Go to your repository on GitHub in a browser.
- You should now see **file1.py** and **file2.py** listed.

Understanding Staging & Committing

🚀 Git Workflow: From Local to Remote Repository





H0. Create, Save, and Push a New Python File to GitHub



Objective

- Practice the complete workflow of cloning a repository, adding a new file, and pushing it to GitHub using VS Code.



Instructions:

- **Clone your GitHub repository** to your local machine using Git Bash.
- **Open** the cloned folder in VS Code.
- **Create a new Python file** called **third_file.py**.
- Add the following code and **save** the file:

```
print("This is my third Python file!")
```

- **Stage and commit** the new file with a meaningful message.
- **Push** the changes to your GitHub repository.
- **Verify** that the file appears in your GitHub repo online.

 Steps to Complete the Task

1- Clone the Repository:

- Open Git Bash.
- Run the command:

```
git clone <repository-url>
```

- Replace **<repository-url>** with your repo's actual URL (you can find this on your GitHub repo page by clicking the green "Code" button).

2- Open in VS Code:

- Navigate into the cloned folder and open it in VS Code:

```
cd my-first-repo  
code .
```

3- Create the Python File:

- In the file explorer (left panel in VS Code), right-click and choose **New File**.
- Name it **third_file.py**.

4- Write and Save Code:

- Add this line inside the file:

```
print("This is my third Python file!")
```

- Save the file with **Ctrl + S** (Windows) or **Cmd + S** (Mac).

5- Stage and Commit:

- Open the **Source Control** tab in VS Code (Git icon).
- You'll see **third_file.py** under Changes.
- Click the + **icon** to stage it.
- In the message box at the top, type:

```
print("This is my third Python file!")
```

- Press **Ctrl + Enter** (Windows) or **Cmd + Enter** (Mac) to commit.

6- Push to GitHub:

- Click the ... (**ellipsis**) menu in the Source Control tab.
- Select **Push**.

7- Write and Save Code:

- Go to your GitHub repository in the browser.
- Check that **third_file.py** now appears in the file list.

Git Commits

- What is a Commit?
- Why Commit Regularly?
- Viewing Commits in Local Repository (Git Graph)
- Viewing Commits in Remote Repository (GitHub)
- What is a Commit ID?
- how to see commit ID on VS Code
- How to See Commit IDs on a Remote Repository (GitHub)
- Commit Indicators in Git
- Where to See Commit Indicators?

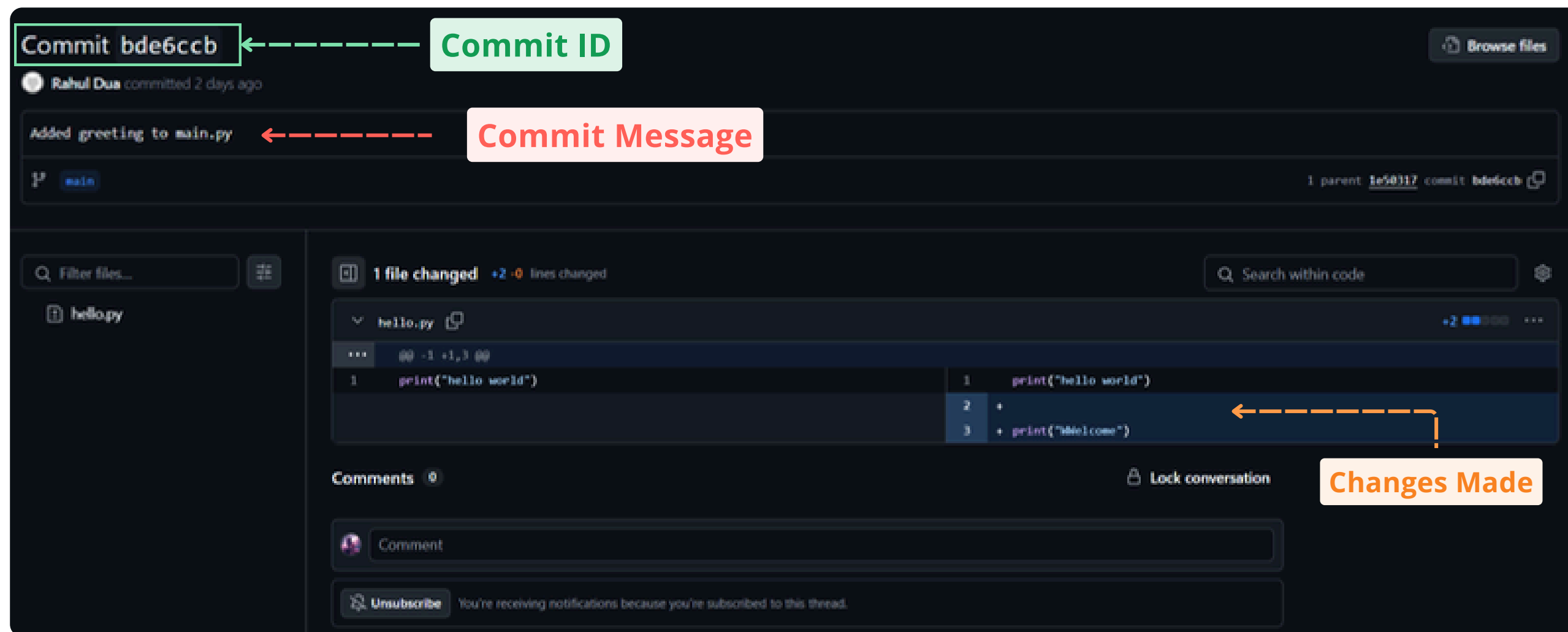
What is a Commit?

Commit

A Snapshot of Changes

It Includes

- ✓ **Changes** made to files (added, edited, or deleted).
- ✓ A **commit message** that describes what changes were made.



Why Commit Regularly?



Track Progress

Helps you track what changes were made & when.

Revert Changes

Enables you to go back to a previous version if needed.

Collaboration

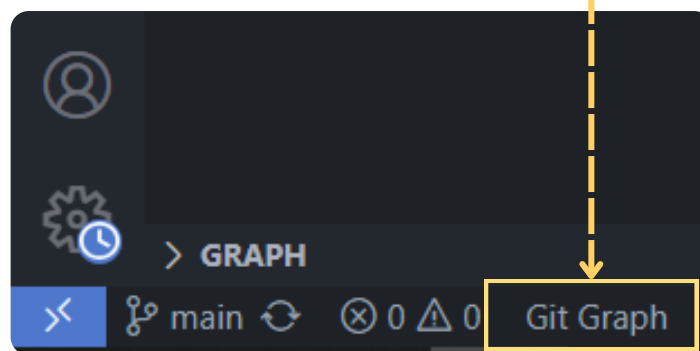
Provides a clear history for team members to understand updates.



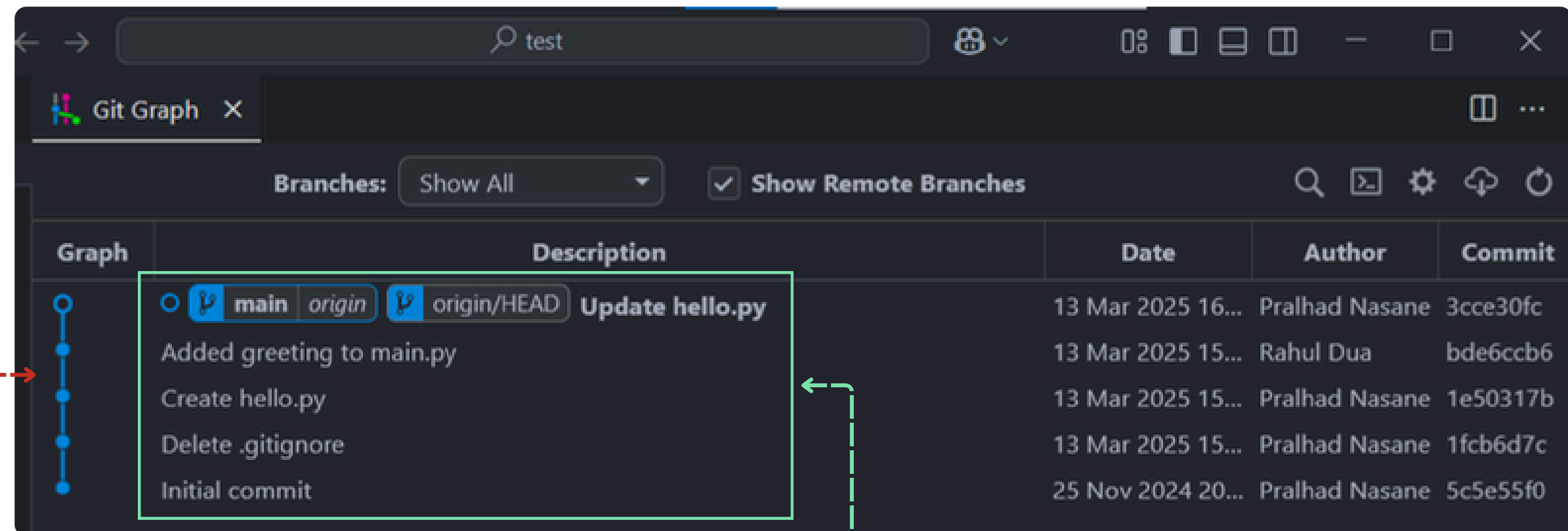
Viewing Commits in Local Repository (Git Graph)

Git Graph is a **VS Code extension** that visually represents your **commit history** and **branches**.

Click the Git Graph icon



A **graphical view** of your **commits will appear**, showing **branches**, **merges**, and **commit history**.



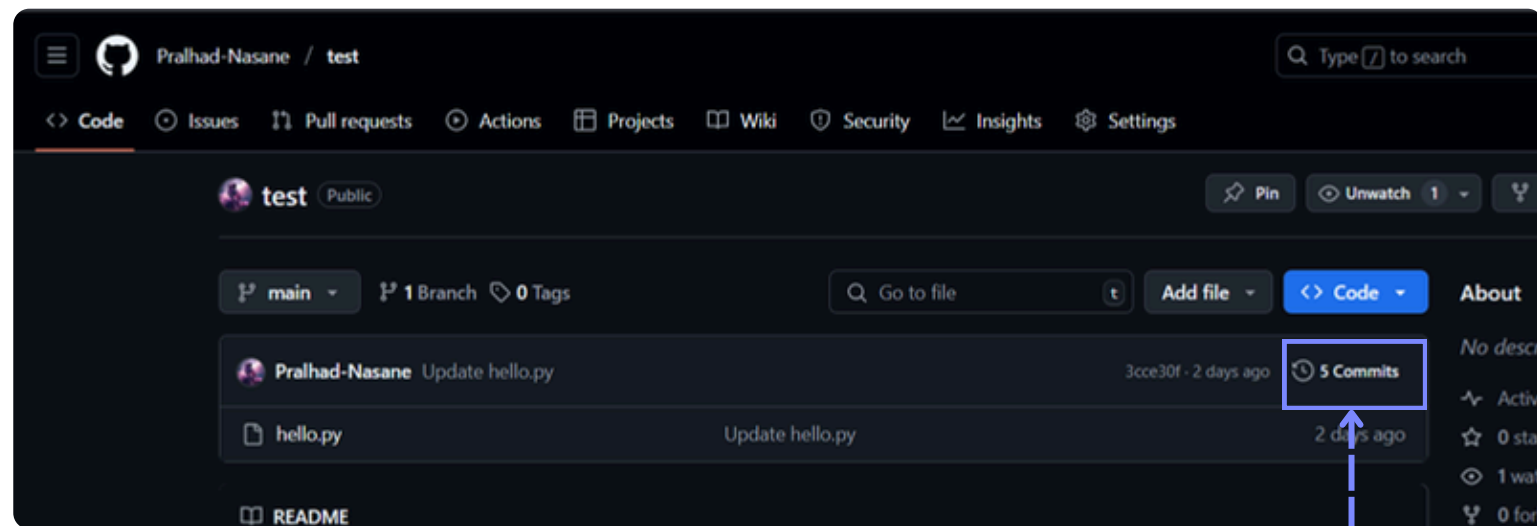
Main Branch

Commit message

Commit ID

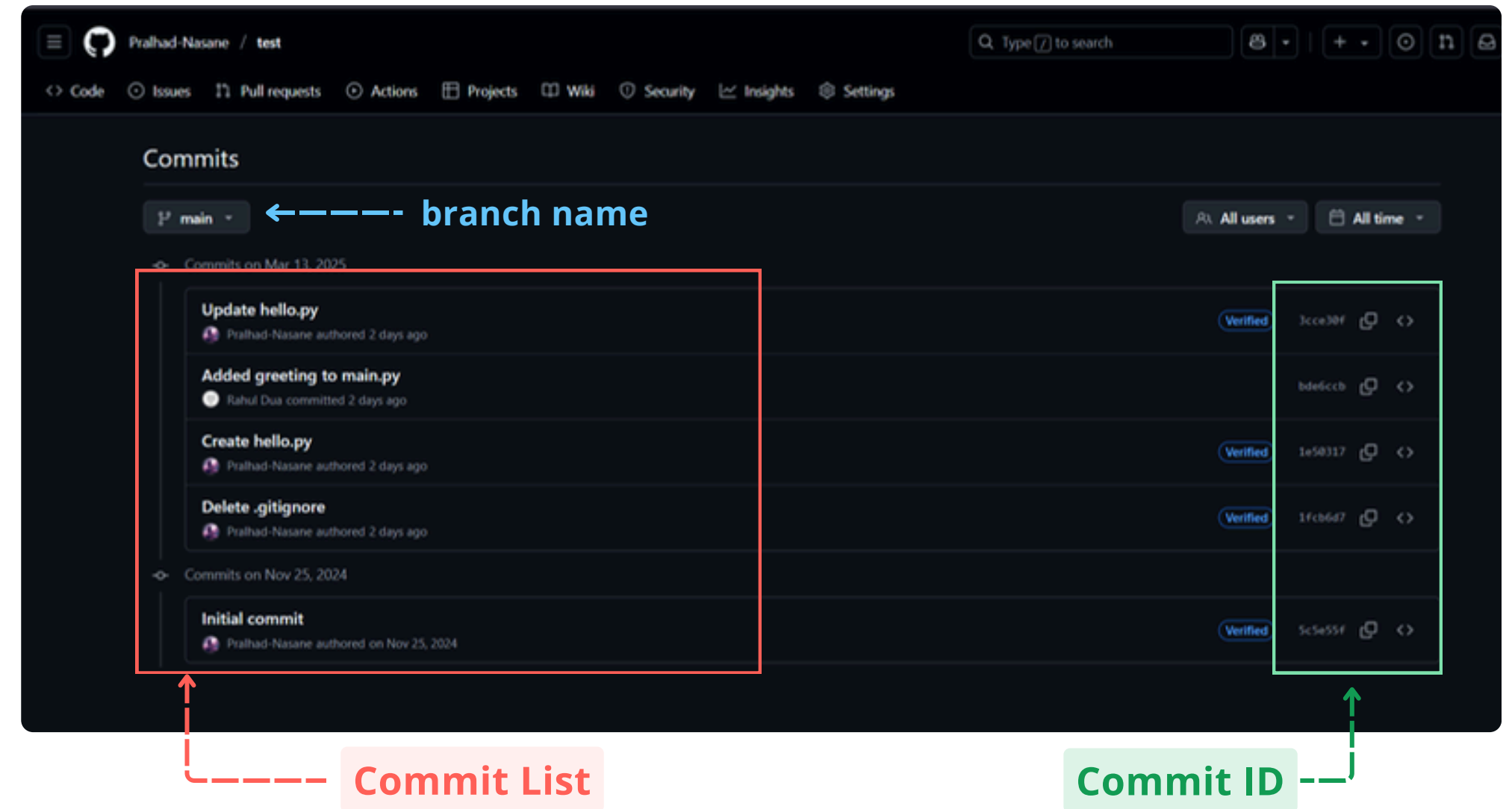
Viewing Commits in Remote Repository (GitHub)

Open **GitHub** in your web browser.
Navigate to your **repository page**.



click on commit

📌 Clicking on a commit displays the changes made in that commit.



What is a Commit ID?

Commit `bde6ccb`

Commit `3cce30f`

Commit `1e50317`

What is a Commit ID?

- **unique alphanumeric identifier (SHA hash)** that Git generates for every commit.
- It ensures each **commit is uniquely identifiable** within a **repository**.

◆ Example of Full Commit ID:

`3f4e2b768d9ab3f0c5e9f2c6ad59c2d4a1b8c9ed`

Purpose of Commit IDs

Tracks changes in the repository

✓ References specific commits for

- **debugging** ,
- **rollbacks** , or
- **collaboration**

how to see commit ID on VS Code

👉 "Open Git Graph by clicking the Git Graph icon at the bottom left of VS Code."

The screenshot shows the Git Graph interface in VS Code. The left sidebar displays a commit graph with branches (main, origin, origin/HEAD) and commit messages. The main area shows a detailed view of a commit, including its SHA hash, parent commit, author, committer, date, and commit message. The commit message is 'Create hello.py'. The commit details are as follows:

Commit ID (SHA Hash)	Parent Commit(s)	Author & Committer Details	Commit Message & Changes
1e50317b5f6d984e2ca0ad7780ddae0428f2192f	1fcb6d7c86bd9695b7039e6158f31d6101772639	Author: Pralhad Nasane <pralhadspace@gmail.com> Committer: GitHub <noreply@github.com> Date: Thu Mar 13 2025 15:32:21 GMT+0530 (India Standard Time)	Create hello.py
1fcb6d7c86bd9695b7039e6158f31d6101772639	5c5e55f0	Author: Pralhad Nasane <pralhadspace@gmail.com> Committer: Pralhad Nasane <pralhadspace@gmail.com> Date: 25 Nov 2024 20:00:00 GMT+0530 (India Standard Time)	Delete .gitignore Initial commit

- **Commit ID (SHA Hash)**

- A unique identifier for this commit

- **Parent Commit(s)**

- The commit(s) from which this commit is derived

- **Author & Committer Details**

- Shows who created and committed the changes

- **Commit Message & Changes**

- Describes what changes were made in this commit

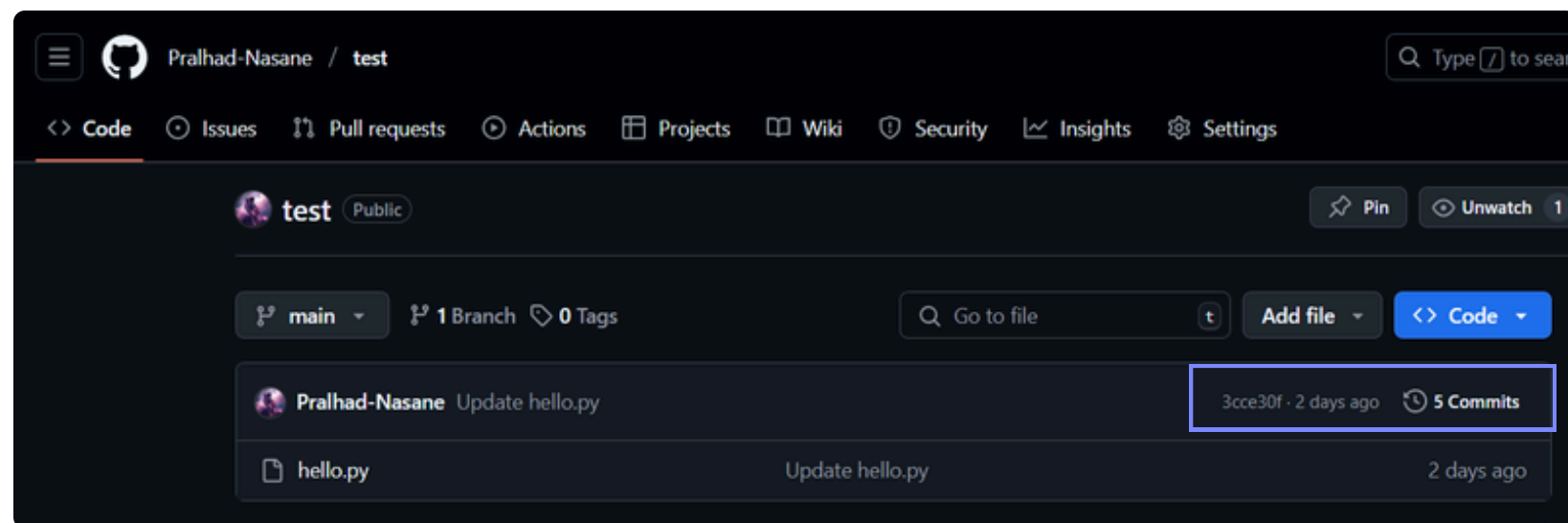
How to See Commit IDs on a Remote Repository (GitHub)

Navigate to the Repository

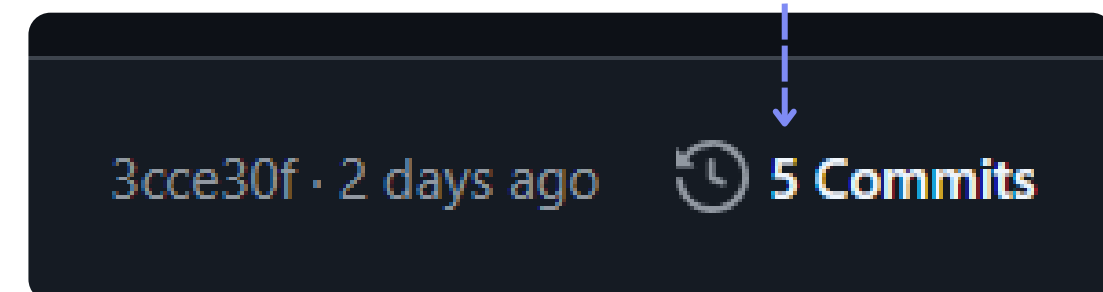
- Open your web browser
- Go to GitHub and open your repository

Access the Commit History

- Click on the "**Commits**" link
- Found below the **Code Button**

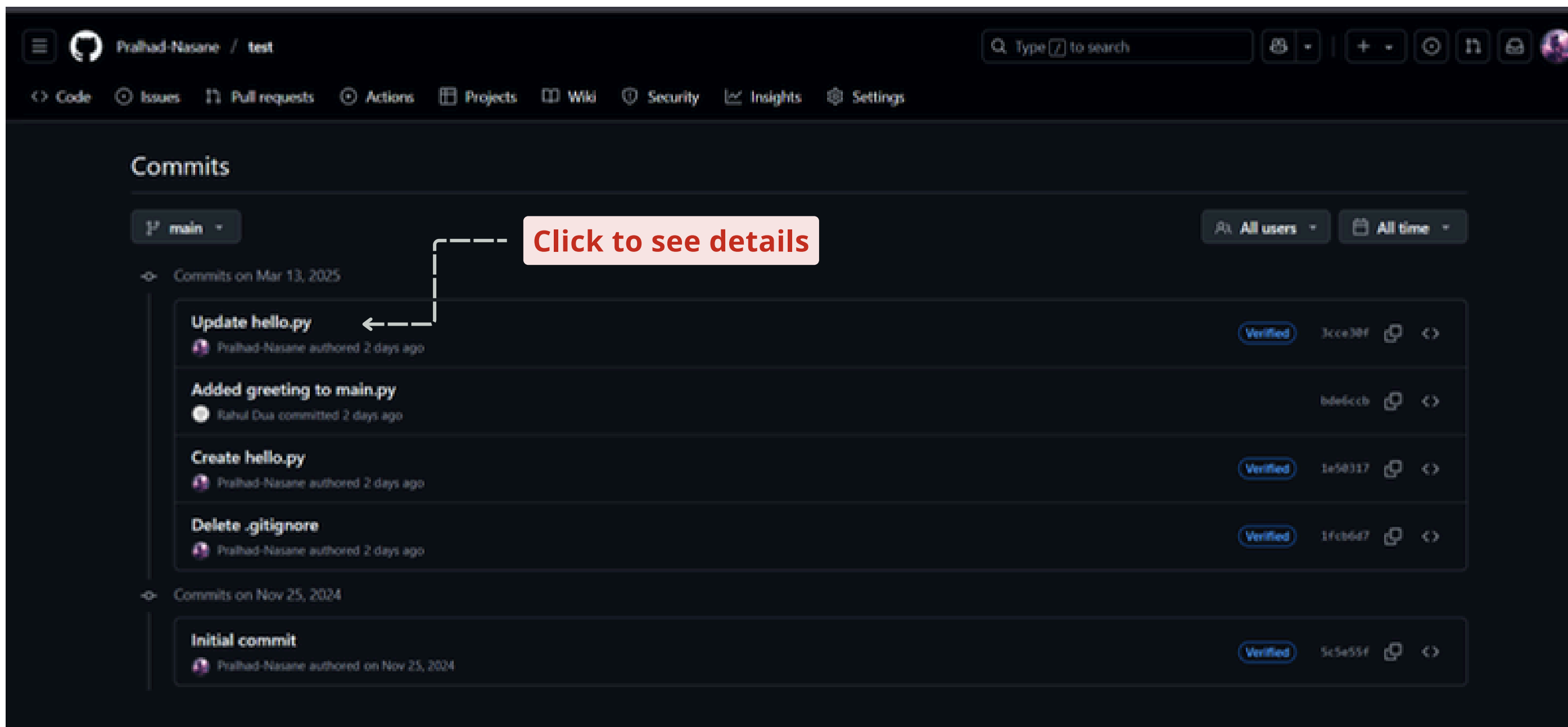


It Shows Number of Commits



See the Full Commit

- Click on a **commit to open its details**.
- The full Commit ID (SHA hash) will be displayed



View Commit IDs

The commit page URL also contains the full Commit

Commit 3cce30f

Update hello.py

Commit message – Describes what was changed.

1 file changed +0 -2 lines changed

```
hello.py
---
+++
@@ -1,3 +1 @@
1 print("hello world")
2 -
3 - print("Welcome")
```

Comments 0

Unsubscribe You're receiving notifications because you're subscribed to this thread.

✓ **Short Commit ID –**
Shows the first 7-10 characters of the full SHA hash.

Commit Indicators in Git

- Git provides **color-coded indicators** to help **visualize changes in a commit**.
- This makes it easier to **track modifications**.

```
file_01.py  file_02.py  file_03.py  Git Graph  file_03.py (b060d3bb^ ← b060d3bb) X
```

```
1 print("Hello from file3!")
2 print("Hello from file3!")
3- print("Hello from file3!")
4- print("Hello from file3!")
5- print("Hello from file3!")
6 print("Hello from file3!")
```

```
1 print("Hello from file3!")
2 print("Hello from file3!")
3+ 
4 print("Hello from file3!")
```

■ **Red** = Removed lines

■ **Green** = Added lines

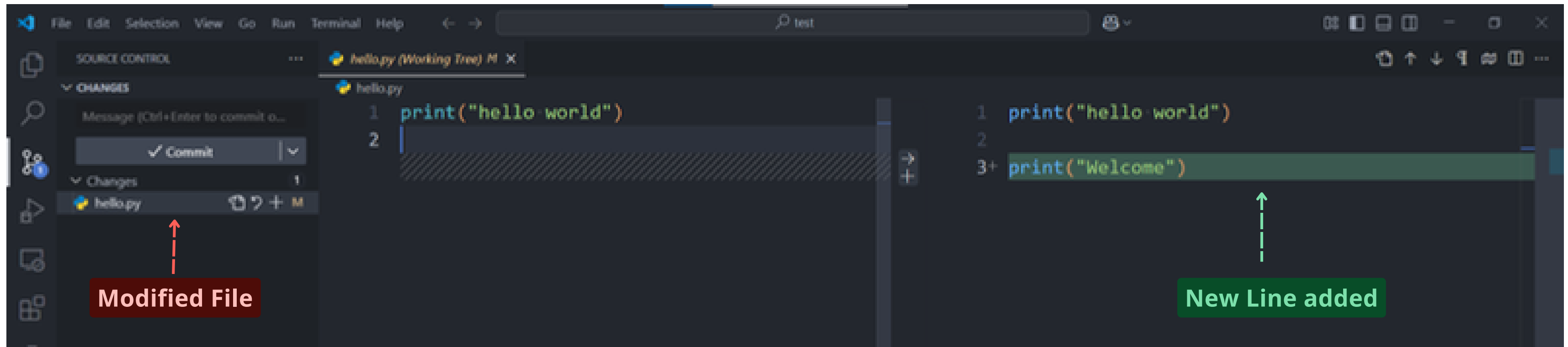
◆ Why Are Commit Indicators Useful?

- ✓ **Understand code evolution** by showing what changed.
- ✓ Makes it easier to **review modifications before committing**.
- ✓ Helpful for **code reviews** and **collaboration**.

Where to See Commit Indicators?

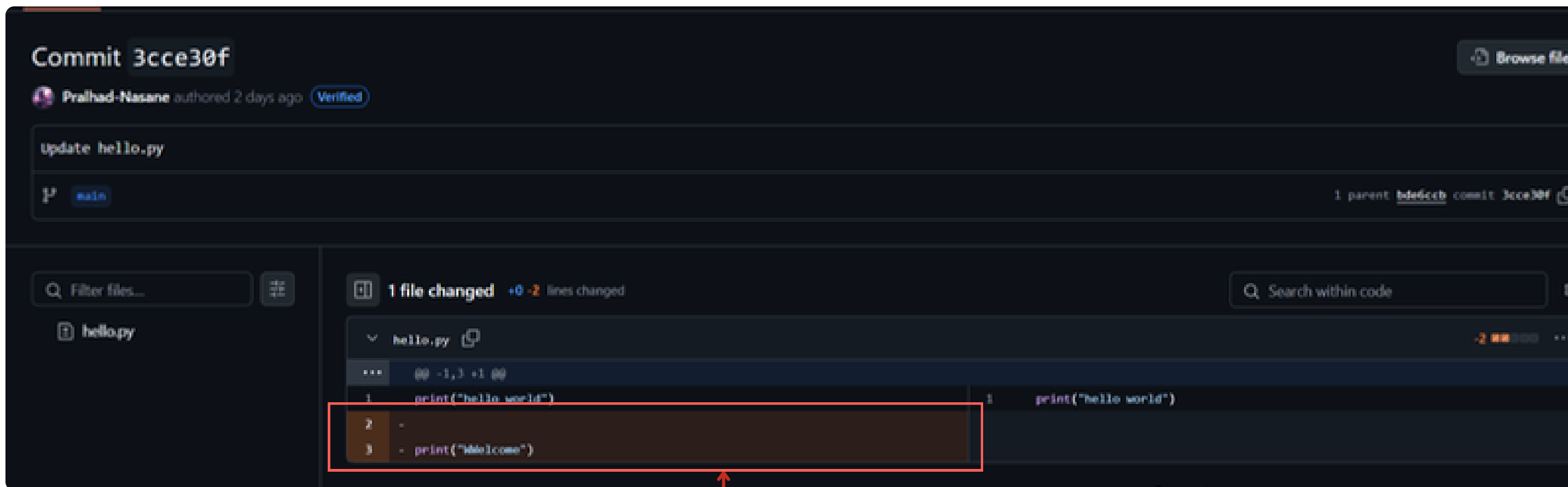
In VS Code

- Open the **Source Control** panel (Ctrl + Shift + G).
- Click on a **modified file** to see commit indicator



On GitHub

- Open a **commit** in your repository.
- GitHub will show the **added (green)** and **removed (red)** lines in the commit history.



The screenshot shows a GitHub commit page for commit 3cce30f. The commit message is "update hello.py". The file "hello.py" is shown with 1 file changed and +0 -2 lines changed. The code diff for "hello.py" is displayed, showing two versions of the file. The left version (parent commit) has two lines: "print('hello world!')" and "print('Welcome!')". The right version (current commit) has one line: "print('hello world!')". The line "print('Welcome!')" is highlighted in red in the left version, indicating it was removed. A red dashed arrow points from the legend below to this red line.

```
... @@ -1,3 +1 @@
1   print("hello world!")
2   -
3   - print("Welcome!")
1   print("hello world!")
```

■ Red = Removed lines



H0: Practice the Complete Process – Edit, Commit, and Push to GitHub



Objective

- Practice Git workflow by **editing an existing file, committing the changes, and pushing** them to your GitHub repository.



Instructions:

To complete this task:

- Open one of your Python files (e.g., **file1.py**) in VS Code.
- **Edit the file** by adding or changing code.
- **Save** the file.
- **Stage and commit** the changes with a clear message.
- **Push** the updated file to GitHub.
- **Verify** that your changes appear in the remote repository.



Steps to Complete the Task

1- Edit the File:

- In VS Code, open **file1.py** (or any other file from your repo).
- Add or change the code. For example:

```
print("Hello, World!")  
print("This is a change!")
```

- Save with **Ctrl + S** (Windows) or **Cmd + S** (Mac), or via **File > Save**.

2- Stage the Changes:

- Go to the **Source Control** tab (Git icon on the sidebar).
- Your edited file will appear under Changes.
- Click the + icon next to the file to stage it—or use the one next to Changes to stage all.

3- Commit the Changes:

- Type a commit message like:

```
Updated print statements
```

- Click the ✓ icon or press **Ctrl + Enter** / **Cmd + Enter** to commit.

4- Push to GitHub:

- In the Source Control tab, click the ... (**ellipsis**) in the top-right.
- Select **Push** to send your commit to the remote repository.

5- Verify on GitHub:

- Open your GitHub repository in a browser.
- Navigate to the file you edited.
- Confirm that your updates are visible.



H0: Create and Manage Own Python Basics Repository



Objective

- Get hands-on practice with Git and GitHub by **creating a personal Python practice repository**.
- You'll clone it using **Git Bash** and manage files and **version control** using **VS Code's user interface**.



Instructions:

To complete this task:

- Create a **new GitHub repository** named something like **python_basics_practice**.
- **Clone** the repository to your local machine using Git Bash.
- **Open** the cloned folder in VS Code.
- Create separate Python files to practice topics covered in your Python lessons.
- Use the **Source Control tab** in VS Code to **stage, commit, and push** your work.
- **Verify** the changes appear in your GitHub repo.



Steps to Complete the Task

1- Create Your Own GitHub Repository:

- Log in to your GitHub account.
- Create a new repository named something like **python_basics_practice**.
- Set it as **Public** or **Private**, as you prefer.
- Add a **README.md** file to initialize the repository.

2- Clone the Repository Using Git Bash:

- Open **Git Bash** in the folder where you want to clone the repository.
- Use the command:

```
git clone <repository_url>
```

3- Open the Repository in VS Code:

- Launch **VS Code**.
- Open the cloned folder by going to **File > Open Folder** and selecting the repository folder.

4- Start Practicing Python Basics:

- In your repository folder, create a new Python file for each topic. [python task provided during python lecture]
- Example file names:
 - **printing.py** for practicing printing text.
 - **variables.py** for variables and data types.
 - **loops.py** for loops and conditions.

5- Review Changes in VS Code:

- Click the **Source Control** icon on the left sidebar (or press **Ctrl + Shift + G**).
- VS Code will display a list of modified files under **Changes**.

6- Stage and Commit Changes Using VS Code UI:

- **Stage Files:**
 - Hover over the file name under Changes and click the + icon to stage it.
 - Alternatively, click on Stage All Changes to stage all files at once.
- **Commit Changes:**
 - Write a descriptive commit message (e.g., "Added basics.py with initial Python practice") in the message box at the top.
 - Click the ✓ Commit icon to save the changes locally.

7- Push Changes to GitHub:

- Click the ... (**ellipsis**) icon in the Source Control view.
 - Select **Push** from the menu to upload your committed changes to the remote repository.
 - VS Code will sync the changes with your GitHub repository.
- 

Thank You

